IMT Atlantique
Projet 28
Campus de Brest

UI AI Natives – Réinventer l'interaction homme-machine à travers les Generative UIs

État de l'art

Version 1
Date 17/10/2026

Auteurs :
BADSSI Chaimae
BARET Alexis
BEDOUIN Arnaud
BELKHIRIA Zeineb
MOREAU Maxandre
SAFTA Yassine

Encadré par : STOENESCU Daniel

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Table des matières

1 Agent – Interface **3**

1.1 A2UI . . . 3

1.2 AG-UI . . . 4

2 Agent – Outils **5**

2.1 Function Calling . . . 5

2.2 MCP . . . 6

2.3 WebMCP . . . 7

2.4 UTCP . . . 8

2.5 Synthèse . . . 9

3 Agent – Agent **10**

3.1 A2A . . . 10

4 Frameworks front-end **11**

4.1 Frameworks étudiés . . . 11

4.2 Compatibilité avec les bibliothèques d'interface générative . . . 12

5 Frameworks d'interfaces génératives **13**

5.1 CopilotKit . . . 13

5.2 Vercel AI SDK . . . 14

5.3 assistant-ui . . . 15

5.4 Tambo . . . 16

5.5 Thesys C1 et OpenUI . . . 17

5.6 Synthèse . . . 19

6 Frameworks d'orchestration **19**

6.1 Indépendants . . . 19

6.1.1 LangChain, LangGraph, LangSmith . . . 19

6.1.2 Mastra . . . 20

6.1.3 CrewAI . . . 21

6.1.4 Pydantic AI . . . 21

6.2 Liés à un fournisseur de modèles . . . 22

6.2.1 OpenAI Agents SDK . . . 22

6.2.2 Claude Agent SDK . . . 22

6.3 Liés à un fournisseur cloud . . . 23

6.3.1 Google ADK . . . 23
6.3.2 Microsoft Agent Framework . . . 23
6.3.3 AWS Strands Agents . . . 24
6.4 Écosystème Java . . . 24
6.4.1 Spring AI . . . 24
6.5 Synthèse et comparaison . . . 25
6.5.1 Caractéristiques générales . . . 25
6.5.2 Protocole des essais pratiques . . . 25
6.5.3 Résultats . . . 27
6.5.4 Comparaison selon les critères retenus . . . 28

7 Observabilité 30
7.1 Traces et standards . . . 30
7.2 Outils . . . 30
7.3 Essai : traces conservées en local . . . 31

8 Synthèse 33
8.1 Comment ces technologies collaborent . . . 33
8.2 Exemple d'architecture . . . 35

9 Choix technologiques 38
9.1 Ce que les essais permettent de conclure . . . 38
9.2 Description de l'interface : A2UI . . . 39
9.3 Agent – interface : AG-UI . . . 39
9.4 Agent – outils : function calling et MCP . . . 40
9.5 Agent – agent : A2A, si nécessaire . . . 40
9.6 Framework front-end : React . . . 40
9.7 Interface générative : CopilotKit . . . 41
9.8 Orchestration : LangChain . . . 41
9.9 Observabilité : LangSmith . . . 42
9.10 Modèle : OpenAI . . . 43

10 Conclusion 43

1 Agent – Interface

1.1 A2UI

A2UI (*Agent-to-User Interface*) est un format ouvert, initié par Google, qui permet à un agent de décrire une interface. L'agent n'écrit ni HTML ni JavaScript : il envoie des messages JSON qui désignent des composants et les données à y afficher. C'est l'application qui dessine ces composants, avec ses propres éléments d'interface et sa propre charte graphique.

Le format repose sur trois principes.

— **Un catalogue fermé.** L'agent ne peut utiliser que les composants d'un catalogue connu de l'application : texte, carte, ligne, colonne, liste, bouton, champ de saisie, etc. L'application peut remplacer le catalogue de base par le sien. Un composant absent du catalogue ne peut pas être affiché.
— **Une liste plate de composants.** Une interface est une liste de composants identifiés, qui se réfèrent les uns aux autres par leur identifiant, et non un arbre imbriqué. Cette forme est plus simple à produire pour un modèle et permet d'afficher l'interface au fur et à mesure de sa génération.
— **Des données séparées de la structure.** Les composants ne contiennent pas nécessairement leurs valeurs : ils peuvent pointer vers un modèle de données, que l'agent met à jour sans renvoyer l'interface. Un composant peut servir de gabarit, répété pour chaque élément d'une liste.

Une interface affichée est appelée une *surface*. L'agent la pilote avec quatre messages :

— `createSurface` crée la surface et désigne son catalogue ;
— `updateComponents` en fournit les composants ;
— `updateDataModel` en fournit ou en modifie les données ;
— `deleteSurface` la supprime.

L'exemple suivant décrit une carte qui affiche un solde.

```
{"version": "v0.9", "createSurface": {"surfaceId": "solde",
                                       "catalogId": "..."}}

{"version": "v0.9", "updateComponents": {"surfaceId": "solde", "components": [
  {"id": "root", "component": "Card", "child": "contenu"},
  {"id": "contenu", "component": "Column", "children": ["titre", "montant"]},
  {"id": "titre", "component": "Text", "text": "Compte courant"},
  {"id": "montant", "component": "Text", "text": {"path": "/solde"}}
]}}

{"version": "v0.9", "updateDataModel": {"surfaceId": "solde", "path": "/",
                                         "value": {"solde": "1 250 €"}}}
```

Dans l'autre sens, lorsque l'utilisateur agit sur un composant, par exemple en appuyant sur un bouton, l'application renvoie à l'agent un message d'action qui porte le nom de l'action et les valeurs saisies.

A2UI ne définit que le contenu des messages, pas leur transport : ils peuvent être acheminés par A2A ou par AG-UI. Le rendu est assuré par une bibliothèque côté application ; il en existe notamment pour le Web et pour Flutter, et des frameworks comme CopilotKit l'intègrent. La version courante de la spécification est la 0.9, la version 1.0 étant en cours de finalisation.

Dans un contexte bancaire, l'intérêt principal d'A2UI est la sécurité : le modèle ne produit aucun

code exécuté par le navigateur, et l'interface reste bornée aux composants que l'application a choisi d'autoriser.

Source : `https://a2ui.org/`

1.2 AG-UI

AG-UI (*Agent–User Interaction Protocol*) est un protocole événementiel ouvert et léger qui standardise la manière dont les agents IA se connectent aux interfaces utilisateur. Il établit une connexion bidirectionnelle entre l'application et l'agent côté serveur, en s'appuyant sur les transports habituels du Web (HTTP, WebSockets). Il normalise trois éléments qui circulent entre les deux : l'état de l'agent, ce qu'il souhaite afficher et les interactions de l'utilisateur.

Sur le Web traditionnel, un échange suit toujours le même schéma : le navigateur envoie une requête, le serveur calcule et renvoie une réponse complète, le navigateur l'affiche, et l'échange est terminé. Les agents ne fonctionnent pas ainsi. D'une part, ils travaillent longtemps et diffusent leur progression au fur et à mesure. D'autre part, ils sont non déterministes : une même demande peut conduire à des parcours différents, si bien que l'interface ne peut pas savoir à l'avance ce qu'elle recevra.

Il faut donc un vocabulaire commun pour que l'agent annonce, pendant l'exécution, ce qui est en train de se passer. AG-UI le fournit sous la forme d'événements typés, que l'interface peut distinguer et traiter différemment : afficher le texte dans la conversation, transformer le résultat d'un outil en graphique, mettre à jour la carte d'un virement.

Le protocole couvre notamment :

— le streaming des réponses ;
— les événements du cycle de vie de l'agent ;
— la synchronisation d'état ;
— les appels d'outils ;
— les interactions entre le frontend et l'agent ;
— la validation humaine (*human-in-the-loop*) et les interruptions ;
— les données multimodales.

Côté serveur, AG-UI peut être utilisé directement avec un LLM ou avec un framework d'orchestration (LangGraph, CrewAI, Google ADK, Mastra…). Côté interface, il est notamment pris en charge par CopilotKit. Il peut enfin transporter des descriptions d'interface au format A2UI, ce qui rend les deux protocoles complémentaires.

Exemple (test AG-UI). Le serveur ci-dessous, écrit avec FastAPI et le SDK Python d'AG-UI, répond à une requête par un flux d'événements typés : le début de l'exécution, un message texte découpé en trois événements (début, contenu, fin), puis la fin de l'exécution. Aucun modèle n'est appelé : l'essai sert uniquement à observer le format des événements échangés.

```
@app.post("/")
async def run_agent(input_data: RunAgentInput, request: Request):
    encoder = EventEncoder(accept=request.headers.get("accept"))

    async def event_stream():
        yield encoder.encode(RunStartedEvent(
            type=EventType.RUN_STARTED,
            thread_id=input_data.thread_id,
            run_id=input_data.run_id,
        ))
```

```
        message_id = str(uuid.uuid4())
        yield encoder.encode(TextMessageStartEvent(
            type=EventType.TEXT_MESSAGE_START,
            message_id=message_id,
            role="assistant",
        ))
        yield encoder.encode(TextMessageContentEvent(
            type=EventType.TEXT_MESSAGE_CONTENT,
            message_id=message_id,
            delta="Votre solde est de 1 250 €.",
        ))
        yield encoder.encode(TextMessageEndEvent(
            type=EventType.TEXT_MESSAGE_END,
            message_id=message_id,
        ))

        yield encoder.encode(RunFinishedEvent(
            type=EventType.RUN_FINISHED,
            thread_id=input_data.thread_id,
            run_id=input_data.run_id,
        ))

    return StreamingResponse(event_stream(),
                             media_type=encoder.get_content_type())
```

Source : `https://docs.ag-ui.com/introduction`

2 Agent – Outils

Un modèle de langage ne produit que du texte : pour consulter un solde ou préparer un virement, il doit pouvoir déclencher du code de l'application. Les technologies de cette section répondent à deux questions distinctes : comment le modèle demande l'exécution d'un outil (function calling), et comment les outils sont décrits et mis à disposition de l'agent (MCP, WebMCP, UTCP).

2.1 Function Calling

Le *function calling* (ou *tool use*) est le mécanisme, proposé par les fournisseurs de modèles dans leur API, qui permet à un modèle de demander l'appel d'une fonction au lieu de répondre par du texte. Le modèle n'exécute jamais la fonction lui-même : il produit une demande structurée, que l'application exécute avant de lui renvoyer le résultat.

L'échange suit quatre étapes :

1. l'application déclare ses fonctions : un nom, une description et le schéma JSON de leurs paramètres ;
2. elle envoie au modèle la conversation accompagnée de ces déclarations ;
3. le modèle répond soit par un texte, soit par une ou plusieurs demandes d'appel, avec les arguments qu'il a choisis ;
4. l'application exécute les fonctions demandées et renvoie leurs résultats au modèle, qui peut alors répondre ou demander d'autres appels.

Le modèle peut demander plusieurs appels indépendants dans une même réponse (appels parallèles) ou enchaîner des appels dont chacun dépend du résultat du précédent. L'application peut aussi encadrer son choix : avec l'API Gemini, le mode `AUTO` (par défaut) laisse le modèle

décider, `ANY` l'oblige à appeler une fonction, `NONE` le lui interdit et `VALIDATED` garantit le respect du schéma déclaré.

C'est cette boucle que les frameworks d'orchestration (section 6) automatisent : ils convertissent les fonctions du code en déclarations, exécutent les appels demandés et relancent le modèle jusqu'à la réponse finale.

Exemple (test function calling). Le script `experiments/function-calling/boucle.py` reproduit cette boucle sans framework, avec le SDK `google-genai`, les deux outils des essais pratiques et la question « Quel est mon solde courant, et quelles dépenses dépassent 500 € ? ». L'exécution automatique des fonctions, que le SDK Python propose, est désactivée pour afficher chaque étape :

```
=== Appel 1 au modèle (151 tokens en entrée, 35 en sortie)
Demande d'appel : get_balance({"account": "courant"})
Résultat renvoyé : '1 250 €'
Demande d'appel : get_transactions({"min_amount": 500})
Résultat renvoyé : 'Loyer : 780 €\nGarage : 620 €\nBillets de train : 510 €'

=== Appel 2 au modèle (243 tokens en entrée, 71 en sortie)
Réponse finale :
Votre solde courant est de **1 250 €**.
Voici les dépenses qui dépassent 500 € au cours des trois derniers mois : ...
```

Le modèle demande les deux appels dans une même réponse (appels parallèles), avec un argument numérique extrait de la question. Au second appel, il reçoit la conversation complète, résultats compris, et rédige la réponse. Ce scénario consomme 394 tokens en entrée au total, contre 392 avec LangChain (tableau 9) : pour un agent simple, le framework n'ajoute presque rien à la requête.

Source : `https://ai.google.dev/gemini-api/docs/function-calling`

2.2 MCP

MCP (*Model Context Protocol*) est un protocole ouvert, initié par Anthropic, qui standardise la manière dont une application d'IA accède à des outils et à des données externes. Là où le function calling décrit l'échange entre l'application et le modèle, MCP décrit l'échange entre l'application et les fournisseurs d'outils : un outil écrit une fois sous forme de serveur MCP est utilisable par toute application compatible.

- **Participants.** L'application d'IA (l'*hôte*) crée un *client* MCP pour chaque *serveur* MCP auquel elle se connecte. Un serveur peut être local ou distant.
- **Primitives.** Un serveur expose des *outils* (fonctions exécutables), des *ressources* (données de contexte) et des *prompts* (modèles de requêtes réutilisables). Le client découvre les outils par `tools/list` et les appelle par `tools/call`. Côté client, l'*élicitation* permet au serveur de demander une information ou une confirmation à l'utilisateur.
- **Couches.** Les messages suivent JSON-RPC 2.0 et circulent par deux transports : `stdio`, pour un serveur lancé en local par l'application, et *Streamable HTTP*, pour un serveur distant, avec une authentification recommandée par OAuth.

La version courante de la spécification est datée du 28 juillet 2026. Elle rend le protocole sans état : chaque requête porte la version du protocole et les capacités du client, et le serveur ex-

pose une requête de découverte (`server/discover`). Elle déclare aussi obsolète l'échantillonnage (*sampling*), par lequel un serveur pouvait demander une génération au modèle du client.

Exemple (test MCP). Les deux outils bancaires des essais pratiques sont exposés par un serveur MCP écrit avec le SDK Python officiel (`experiments/mcp/serveur.py`) :

```
mcp = FastMCP("banque")

@mcp.tool()
def get_transactions(min_amount: float) -> str:
    """Renvoie les transactions des trois derniers mois supérieures à un
       montant en euros."""
    ...

mcp.run()  # transport stdio
```

Un client sans modèle (`client.py`) lance ce serveur, liste ses outils et en appelle un. Le schéma des paramètres est déduit des annotations de type de la fonction Python :

```
Serveur : banque | protocole 2025-11-25
Outil get_transactions : Renvoie les transactions des trois derniers mois ...
{"properties": {"min_amount": {"title": "Min Amount", "type": "number"}},
 "required": ["min_amount"], "type": "object", ...}
Appel de get_transactions(min_amount=500) :
Loyer : 780 €
Garage : 620 €
Billets de train : 510 €
```

Le SDK Python utilisé (version 1.30) négocie la version 2025-11-25 du protocole et non la plus récente : l'écosystème suit la spécification avec un décalage, qu'il faut vérifier pour chaque SDK.

Un agent LangChain (`experiments/mcp/agent.py`) se connecte ensuite au même serveur par l'adaptateur `langchain-mcp-adapters` : il ne définit aucun outil lui-même, mais les découvre au démarrage.

```
client = MultiServerMCPClient(
    {"banque": {"transport": "stdio", "command": sys.executable, "args": ["
       serveur.py"]}})
outils = await client.get_tools()   # ['get_balance', 'get_transactions']
agent = create_agent(model=..., tools=outils, system_prompt=...)
```

À la question « Quel est mon solde courant, et quelles dépenses dépassent 500 € ? », l'agent demande les deux outils dans un même appel ; ils sont exécutés par le serveur MCP, et la réponse finale reprend le solde et les trois transactions. Cet essai a été réalisé avec `gemini-3.8-flash`, le quota journalier gratuit de `gemini-3.5-flash-lite` (500 requêtes) étant alors épuisé.

Source : `https://modelcontextprotocol.io/docs/learn/architecture`

2.3 WebMCP

WebMCP est une proposition de standard web qui permet à une page de déclarer des outils utilisables par un agent. Au lieu de deviner les boutons et les champs d'une page, l'agent appelle des fonctions que le site a prévues pour lui. L'explainer présente les pages qui l'utilisent comme des « serveurs MCP intégrés à la page », conçus pour compléter et non remplacer les serveurs

MCP côté serveur.

— **API impérative.** La page enregistre ses outils par la méthode `registerTool()` de `document.modelContext`, avec un nom, une description, un schéma JSON des paramètres et une fonction `execute`. Par défaut, un outil n'est visible que de la page, des documents de même origine et des agents intégrés au navigateur.
— **API déclarative.** Une seconde API, fondée sur l'annotation des formulaires HTML, est explorée séparément ; elle ne fait pas partie de la spécification à ce stade.
— **Exécution dans la page.** L'outil s'exécute dans le contexte de la page, donc avec la session de l'utilisateur, sans dupliquer son authentification sur un autre serveur.

WebMCP est expérimental. Il est incubé dans l'organisation `webmachinelearning` sur GitHub et sa spécification évolue encore. Chrome le propose en *origin trial* à partir de la version 149, ou localement par le drapeau `chrome://flags/#enable-webmcp-testing`. Son usage est soumis à la politique de permissions `tools`, et la documentation de Chrome indique que l'API est d'abord conçue pour un usage dans le navigateur, avec un humain présent.

WebMCP ne transporte rien jusqu'à un serveur : un agent exécuté côté serveur ne peut pas appeler directement ces outils. L'exemple d'architecture (section 8.2) les lui transmet par AG-UI.

Essai. La page de l'exemple d'architecture a été ouverte dans Chrome 155, le drapeau ci-dessus activé. Le navigateur expose alors `document.modelContext`, avec les méthodes `registerTool`, `getTools` et `executeTool` et l'événement `ontoolchange`. Les deux outils de la page (`consulter_comptes` et `consulter_transactions`) figurent dans la liste renvoyée par `getTools`, avec leur description, leur schéma JSON (généré depuis le schéma Zod) et l'origine de la page ; le navigateur complète les annotations (`readOnlyHint` déclaré, plus `consequentialHint` et `untrustedContentHint` à `false`). Appelé par `executeTool`, l'outil s'exécute dans la page : il interroge l'API bancaire et renvoie le résultat, par exemple les deux transactions supérieures à 600 € pour `min_amount = 600`. Ces appels ont été faits par un script exécuté dans la page, qui joue le rôle de l'agent ; aucun agent intégré au navigateur n'a été utilisé.

Source : `https://developer.chrome.com/docs/ai/webmcp`

Source : `https://github.com/webmachinelearning/webmcp`

2.4 UTCP

UTCP (*Universal Tool Calling Protocol*) est un protocole ouvert qui prend le contre-pied de MCP : plutôt que d'envelopper chaque outil dans un serveur dédié, il décrit comment appeler l'outil directement, par son protocole d'origine. Un fournisseur publie un *manuel*, document JSON qui liste ses outils avec, pour chacun, un schéma des paramètres et un *call template* indiquant comment l'appeler (HTTP, ligne de commande, SSE, WebSocket, fichier texte, ou même un serveur MCP).

L'intérêt avancé est de réutiliser l'API et l'authentification existantes, sans serveur intermédiaire ni double saut réseau. La documentation courante est en version 1.1 ; des SDK existent en Python, TypeScript et Go, sous licence Apache 2.0. Le protocole est porté par une communauté de contributeurs et non par un grand éditeur, ce qui le rend nettement moins répandu que MCP.

Exemple (test UTCP). Une API REST ordinaire (`experiments/utcp/api.py`) se voit ajouter une seule route, `/utcp`, qui renvoie le manuel :

```
{"name": "get_transactions",
 "description": "Renvoie les transactions ... supérieures à un montant en
     euros.",
 "inputs": {"type": "object", "properties": {"min_amount": {"type": "number
     "}}},
 "tool_call_template": {"call_template_type": "http",
                        "url": "http://127.0.0.1:8100/transactions",
                        "http_method": "GET"}}
```

Le client UTCP lit ce manuel, puis appelle les outils. Le journal du serveur confirme que les appels arrivent directement sur les routes existantes de l'API :

```
GET /utcp HTTP/1.1                              200 OK
GET /comptes/courant/solde HTTP/1.1             200 OK
GET /transactions?min_amount=500 HTTP/1.1       200 OK
```

Source : `https://www.utcp.io/`

2.5 Synthèse

Technologie	**Frontière**	**Rôle**	**Maturité**
Function calling	Application – modèle	Le modèle demande l'appel d'une fonction déclarée.	Standard de fait, propre à chaque API de modèle
MCP	Application – serveur d'outils	Découverte et appel d'outils exposés par un serveur dédié, local ou distant.	Très répandu, spécification encore en évolution
WebMCP	Page – agent du navigateur	Déclaration d'outils exécutés dans la page, avec la session de l'utilisateur.	Expérimental (*origin trial* Chrome)
UTCP	Application – API existante	Manuel décrivant comment appeler directement des outils existants.	Communautaire, peu répandu

Table 1 – Technologies de la frontière agent – outils.

MCP et WebMCP. Malgré leur nom, WebMCP n'est ni une extension ni un remplacement de MCP : c'est une API du navigateur « inspirée de MCP », qui reprend la notion d'outil mais pas celle de ressource. Les deux visent à exposer les outils d'une application aux agents par des appels de fonction explicites, plutôt que de laisser l'agent deviner l'interface ; ils diffèrent sur les points suivants.

	MCP	**WebMCP**
Exécution	Serveur (backend)	Page ouverte dans le navigateur (frontend)
Durée de vie	Persistante : le serveur tourne en permanence	Éphémère : les outils disparaissent à la fermeture de l'onglet
Utilisateurs	Tout agent, intégré ou non à un navigateur	Agent intégré au navigateur
Découverte	Configuration du serveur auprès de chaque agent	Outils enregistrés par la page lors de la visite
Accès	Données et services du backend	Session, cookies et DOM de l'onglet actif
Interface	Indépendant de la page ; une interface éventuelle s'affiche dans celle de l'agent	L'agent agit dans le site, sous les yeux de l'utilisateur
Usage type	Actions d'API et tâches de fond, sans utilisateur présent	Actions sur la page pendant que l'utilisateur la consulte

Table 2 – Différences entre MCP et WebMCP, d'après la documentation de Chrome.

La documentation de Chrome les présente comme complémentaires : MCP porte la logique métier et l'accès aux données, WebMCP l'interaction avec le site pendant la visite de l'utilisateur.

WebMCP est donc réservé aux agents intégrés au navigateur (assistant du navigateur ou extension), qui agissent sur la page ouverte. Un agent exécuté côté serveur, comme celui d'une application, n'a pas accès à ces outils : il appelle les siens directement ou par l'intermédiaire d'un serveur MCP.

Source : `https://developer.chrome.com/docs/ai/webmcp/compare-mcp`

3 Agent – Agent

3.1 A2A

A2A (*Agent2Agent*) est un protocole ouvert, lancé par Google, qui permet à des agents indépendants, construits avec des frameworks différents et exploités par des organisations différentes, de collaborer. Chaque agent reste opaque : il expose ce qu'il sait faire, sans partager sa mémoire, ses outils ni son fonctionnement interne. A2A est complémentaire de MCP : MCP relie un agent à ses outils, A2A relie des agents entre eux. La version courante de la spécification est la 1.0.

Le protocole repose sur quelques objets :

- **Agent Card.** Document JSON publié par l'agent, qui décrit son identité, ses compétences (*skills*), son adresse, les protocoles qu'il accepte et l'authentification exigée. Il sert à la découverte.
- **Tâche.** Unité de travail confiée à un agent, qui passe par des états définis : soumise, en cours, terminée, en échec, annulée, refusée, ou interrompue dans l'attente d'une saisie (`INPUT_REQUIRED`) ou d'une authentification.
- **Message, Part, Artefact.** Les messages portent les échanges, découpés en parties (texte, fichier, données structurées) ; les résultats d'une tâche sont rendus sous forme d'artefacts.

Les opérations (envoi d'un message, en flux ou non, consultation, liste et annulation de tâches, notifications par webhook) sont disponibles par trois liaisons équivalentes : JSON-RPC, gRPC et HTTP+JSON. L'authentification suit les schémas déclarés dans l'Agent Card (clé d'API, OAuth 2.0, OpenID Connect, TLS mutuel).

Exemple (test A2A). Un agent « comptes » sans modèle est exposé avec le SDK Python officiel (`experiments/a2a/serveur.py`). Son Agent Card est servie à l'adresse `/.well-known/agent-card.json` :

```
{"name": "Agent comptes",
 "description": "Donne le solde des comptes du client (données fictives).",
 "supportedInterfaces": [{"url": "http://127.0.0.1:9999",
                          "protocolBinding": "JSONRPC", "protocolVersion":
                              "1.0"}],
 "capabilities": {"streaming": true},
 "skills": [{"id": "soldes", "name": "Soldes", ...}]}
```

Un client lit cette carte, puis confie une tâche à l'agent (« Quel est le solde de mon livret ? ») en mode flux. Il reçoit successivement quatre événements : la tâche à l'état `SUBMITTED`, son passage à `WORKING`, un artefact contenant « livret : 8 400 € », puis l'état `COMPLETED`.

Dans le projet, un seul agent est prévu à ce stade : A2A ne deviendrait utile que si l'assistant devait déléguer à des agents spécialisés, éventuellement gérés par d'autres équipes de la banque.

Source : `https://a2a-protocol.org/latest/specification/`

4 Frameworks front-end

Les bibliothèques d'interface générative présentées à la section 5 ne s'utilisent pas seules : elles s'appuient sur un framework front-end, qui gère les composants de la page, leur état et leur mise à jour. Le choix de ce framework conditionne donc les bibliothèques disponibles.

4.1 Frameworks étudiés

- **React** : bibliothèque de composants développée par Meta. Une interface est une arborescence de composants (fonctions qui renvoient du JSX) ; l'état est géré par des *hooks*.
- **Vue** : framework progressif, porté par une communauté indépendante, fondé sur des composants à fichier unique et un système réactif.
- **Angular** : framework complet développé par Google, en TypeScript, qui fournit d'emblée routage, formulaires, injection de dépendances et client HTTP.
- **Svelte** : framework qui compile les composants en JavaScript au moment de la construction, sans moteur de rendu embarqué.

Ces frameworks sont souvent utilisés à travers un *méta-framework*, qui ajoute le routage, le rendu côté serveur et des routes d'API : Next.js pour React, Nuxt pour Vue, SvelteKit pour Svelte. À l'inverse, une application monopage peut être construite avec un simple outil de construction comme Vite, le serveur étant alors un composant séparé ; c'est le choix fait dans les essais du dossier `experiments/`.

Projet	Nature	Étoiles GitHub	Téléchargements npm (semaine)	Licence
React	Bibliothèque de composants	251k	188 millions	MIT
Vue	Framework	54,7k	17,6 millions	MIT
Angular	Framework complet	101k	6,2 millions	MIT
Svelte	Framework compilé	88,4k	6,7 millions	MIT
Next.js	Méta-framework (React)	143k	67,7 millions	MIT
Nuxt	Méta-framework (Vue)	61k	2,4 millions	MIT
SvelteKit	Méta-framework (Svelte)	20,9k	3,3 millions	MIT
Vite	Outil de construction	83,4k	196 millions	MIT

TABLE 3 – Frameworks front-end. Étoiles relevées en octobre 2026 ; téléchargements npm de la semaine du 5 au 11 octobre 2026 (paquets `react`, `vue`, `@angular/core`, `svelte`, `next`, `nuxt`, `@sveltejs/kit`, `vite`).

4.2 Compatibilité avec les bibliothèques d'interface générative

Le tableau 4 croise ces frameworks avec les bibliothèques de la section 5, d'après leur documentation.

Bibliothèque	React	Vue	Angular	Svelte	Autres
CopilotKit	oui	–	oui	–	
Vercel AI SDK (UI)	oui	oui	oui	oui	
assistant-ui	oui	oui	–	–	React Native, terminal
Tambo	oui	–	–	–	
OpenUI (Thesys)	oui	oui	oui	oui	React Native
Rendu A2UI	oui	–	–	–	Web Components (Lit), Flutter

TABLE 4 – Frameworks front-end pris en charge par les bibliothèques d'interface générative (« – » : non mentionné dans la documentation consultée).

React est le seul framework pris en charge par toutes les bibliothèques étudiées. Vue et Angular réduisent le choix de bibliothèques. La question ouverte est plutôt celle du méta-framework : une application Vite suffit lorsque le serveur est un composant séparé.

Source : `https://react.dev/`

Source : `https://vuejs.org/`

Source : `https://angular.dev/`

Source : `https://svelte.dev/`

5 Frameworks d'interfaces génératives

Les protocoles présentés plus haut définissent ce qui circule entre un agent et une interface. Les frameworks d'interfaces génératives se placent du côté de l'application : ils reçoivent ce que l'agent produit et le transforment en composants affichés à l'écran. Ils fournissent en général la conversation (messages en streaming, saisie, historique), le rendu des appels d'outils et le renvoi des actions de l'utilisateur vers l'agent.

Ils se distinguent d'abord par la liberté laissée au modèle dans la construction de l'interface. Trois approches coexistent (tableau 5), de la plus contrôlée à la plus ouverte.

Approche	**Principe**	**Conséquence**
Statique	Les composants sont écrits par les développeurs. L'agent choisit lequel afficher et fournit ses données, le plus souvent par un appel d'outil.	Rendu entièrement maîtrisé, mais chaque affichage doit avoir été prévu.
Déclarative	L'agent décrit l'interface en données, en assemblant les composants d'un catalogue fermé (A2UI).	Interfaces composées à la demande, sans exécuter de code produit par le modèle.
Ouverte	L'agent ou un serveur d'outils fournit directement du HTML, affiché dans un cadre isolé (MCP Apps).	Liberté maximale, au prix d'un contrôle plus faible sur le rendu et d'une isolation à garantir.

Table 5 – Les trois approches de l'interface générative.

5.1 CopilotKit

CopilotKit est un framework open source pour intégrer des agents dans une application React ou Angular. Son éditeur est à l'origine du protocole AG-UI, sur lequel repose toute la communication entre l'interface et l'agent : tout agent qui parle AG-UI, quel que soit son framework d'orchestration, peut donc être branché sur une interface CopilotKit.

Le framework se compose de deux parties. Côté navigateur, des composants de conversation prêts à l'emploi et des hooks pour partager l'état de l'application avec l'agent ou lui déclarer des outils exécutés dans la page. Côté serveur, un *runtime* qui sert de relais entre la page et les agents. Les trois approches du tableau 5 sont prises en charge : composants de l'application associés aux appels d'outils, surfaces A2UI et MCP Apps. Il est distribué sous licence MIT ; l'éditeur propose en complément une offre hébergée.

Brique	**Rôle**
Composants de conversation	Fenêtre, panneau latéral ou conversation intégrée à la page.
Runtime	Relais entre la page et les agents, auquel s'ajoutent des middlewares (A2UI, MCP Apps).
Outils côté page	Fonctions de l'application que l'agent peut appeler dans le navigateur.
État partagé	Synchronisation de l'état entre l'agent et l'application.
Validation humaine	Interruption de l'agent en attente d'une décision de l'utilisateur.

Source : `https://docs.copilotkit.ai/`

5.2 Vercel AI SDK

L'AI SDK de Vercel est une bibliothèque TypeScript open source pour construire des applications fondées sur des modèles de langage. Elle comprend une partie serveur, qui unifie l'appel des modèles de nombreux fournisseurs et la boucle d'appel d'outils, et une partie interface, qui gère l'état d'une conversation en streaming dans React, Vue, Svelte ou Angular.

L'interface générative y suit l'approche statique : chaque message est une suite de parties typées (texte, appel d'outil, résultat), et l'application associe à chaque outil le composant qui doit l'afficher. Le framework ne fournit pas de composants de conversation : il gère l'état et laisse le rendu à l'application. Il est distribué sous licence Apache 2.0.

Brique	**Rôle**
Appel des modèles	Interface commune à de nombreux fournisseurs, avec appel d'outils et sortie structurée.
État de conversation	Hooks qui gèrent les messages, le streaming et l'envoi.
Parties de message	Découpage typé des messages, sur lequel s'appuie le rendu des outils.

Exemple (test AI SDK UI). L'essai `experiments/genui-ai-sdk` associe un serveur Node, qui exécute un agent `ToolLoopAgent` doté des outils bancaires, et une page React (Vite) qui utilise le hook `useChat`. Chaque message reçu est une liste de parties typées : la page associe à chaque outil son propre composant (carte de solde, liste de transactions), choisi d'après le type de la partie et son état d'avancement.

```
case 'tool-get_balance':
  if (part.state !== 'output-available') return <p>Consultation du solde...</p
      >
  return <div className="carte">Compte {part.output.compte} : {euros(part.
      output.solde)}</div>
```

Le type des messages est déduit de la déclaration de l'agent : les arguments et le résultat de chaque outil sont typés dans la page, et une erreur de nom ou de propriété est détectée à la compilation. Un troisième outil, `prepare_transfer`, est déclaré avec `needsApproval: true` : l'agent s'arrête avant de l'exécuter, la page affiche une demande de confirmation et renvoie la décision de l'utilisateur.

La page a été essayée dans le navigateur (figure 1). À la question sur le solde et les dépenses, les deux outils s'affichent sous forme de cartes, suivies du texte de l'agent. À la demande « Fais un virement de 50 € à Alice. », le serveur émet un événement `tool-approval-request` et s'arrête ; la page affiche une carte de confirmation, et ce n'est qu'après le clic sur « Confirmer » que l'outil s'exécute et que l'agent reprend. Pour ces essais d'interface, le modèle `gemini-3.8-flash` a été utilisé, le quota journalier de `gemini-3.5-flash-lite` étant épuisé.

Source : `https://ai-sdk.dev/docs`



FIGURE 1 – Essai AI SDK UI : outils rendus par des composants React, puis virement exécuté après confirmation de l'utilisateur.

5.3 assistant-ui

assistant-ui est une bibliothèque open source consacrée à l'interface de conversation, pour React principalement (sa documentation mentionne aussi Vue, React Native et le terminal). Elle fournit des composants de bas niveau, sans style imposé, que l'application assemble et habille elle-même, dans l'esprit de shadcn/ui. Elle prend en charge ce qui est délicat à réécrire : streaming, défilement, modification et régénération des messages, pièces jointes.

Elle ne gère pas l'agent : des adaptateurs la relient à l'AI SDK de Vercel, à LangGraph, à Mastra ou à un agent AG-UI. Les appels d'outils peuvent être associés à des composants de l'application, et les surfaces A2UI reçues par AG-UI peuvent être rendues. Elle est distribuée sous licence MIT.

Brique	**Rôle**
Primitives	Composants de conversation de bas niveau, à assembler et à styler.
Adaptateurs	Connexion à l'AI SDK, à LangGraph, à Mastra ou à AG-UI.
Interface d'outil	Composant de l'application associé à un appel d'outil.

Sa documentation décrit aussi un rendu déclaratif : l'agent produit une description JSON de l'interface, et seuls les composants d'une liste fournie par l'application peuvent être affichés, tout nom inconnu étant rejeté.

Exemple (test assistant-ui). L'essai `experiments/assistant-ui` reprend l'agent de l'essai précédent sans le modifier : seule la page change. Elle est construite avec les primitives de la bibliothèque (`ThreadPrimitive`, `MessagePrimitive`, `ComposerPrimitive`) et associe les mêmes composants aux outils par leur nom :

```
// envoie la conversation à /api/chat et relance l'agent après une approbation
const runtime = useChatRuntime({ sendAutomaticallyWhen:
    lastAssistantMessageIsCompleteWithApprovalResponses })
...
<MessagePrimitive.Parts components={{ tools: { by_name: {
  get_balance: Solde, get_transactions: Transactions, prepare_transfer:
      Virement } } }} />
```

La confirmation du virement passe par `respondToApproval`, fourni à chaque composant d'outil. Comme avec `useChat`, il faut indiquer au runtime de relancer l'agent une fois la décision prise (`sendAutomaticallyWhen`) : sans cette option, la carte restait sur « Virement en cours ». Une fois l'option ajoutée, le parcours complet (consultation, puis virement confirmé) a fonctionné dans le navigateur (figure 2). Deux constats : l'interface prête à l'emploi (fenêtre de conversation stylée) s'installe par la ligne de commande de shadcn/ui et suppose Tailwind CSS, alors que les primitives seules fonctionnent sans ; et l'API des outils évolue vite, plusieurs fonctions (`makeAssistantToolUI`, `useAssistantToolUI`) étant déjà dépréciées au profit de *toolkits*. L'application compile ; une fois construite, le code JavaScript de la page pèse environ 274 ko compressés (gzip), contre 116 ko pour la page de l'essai AI SDK, React compris.



FIGURE 2 – Essai assistant-ui : même agent et mêmes composants, conversation construite avec les primitives de la bibliothèque.

Source : `https://www.assistant-ui.com/docs`

5.4 Tambo

Tambo est une boîte à outils open source d'interface générative pour React. Le développeur enregistre ses propres composants en décrivant leurs propriétés par un schéma Zod ; l'agent choisit ensuite le composant adapté à la demande et en produit les propriétés, validées par le schéma. L'approche est donc statique, mais le choix du composant est laissé au modèle.

Contrairement aux bibliothèques précédentes, Tambo inclut l'agent : la boucle d'appel du modèle et des outils est assurée par son propre serveur, hébergé par l'éditeur ou par le client. Le SDK et le serveur sont distribués sous licence MIT.

Brique	Rôle
Composants enregistrés	Composants de l'application décrits par un schéma de leurs propriétés.
Serveur	Boucle d'agent, état des conversations et connexion aux outils MCP.

Le SDK React indique s'appuyer sur le protocole AG-UI pour le streaming. Chaque fil de conversation est rattaché à un utilisateur, identifié par une clé ou un jeton.

Exemple (test Tambo, non exécuté). L'essai `experiments/tambo` enregistre deux composants (carte de solde, liste de transactions), décrits par un schéma Zod de leurs propriétés, et deux outils exécutés dans la page. Il compile (environ 455 ko de JavaScript compressé, React compris), mais n'a pas été lancé : Tambo exige une clé de son service hébergé ou l'installation de son serveur.

```
const COMPOSANTS: TamboComponent[] = [
  { name: 'CarteSolde', description: "Affiche le solde d'un compte.",
    component: CarteSolde, propsSchema: z.object({ compte: z.string(), solde:
        z.number() }) },
  ...
]
<TamboProvider apiKey={...} userKey="client-demo" components={COMPOSANTS}
    tools={OUTILS}>
```

Source : `https://docs.tambo.co/`

5.5 Thesys C1 et OpenUI

C1 est une API commerciale de l'éditeur Thesys. Elle se présente comme un point d'accès compatible avec l'API d'OpenAI : l'application y envoie sa conversation et reçoit, à la place d'un texte, la description d'une interface (graphiques, formulaires, tableaux), qu'un SDK React affiche. L'approche est déclarative, mais la génération de l'interface est déléguée à un service tiers.

La documentation de C1 est désormais présentée comme une archive, conservée pour les équipes qui l'utilisent déjà ; l'éditeur oriente les nouveaux projets vers **OpenUI**, projet open source (licence MIT, environ 11,4k étoiles). OpenUI se présente comme un standard ouvert d'interface générative : le développeur définit sa bibliothèque de composants, OpenUI en tire une consigne pour le modèle, le modèle répond dans un langage compact (OpenUI Lang) et un moteur de rendu affiche chaque composant au fil du flux. Des intégrations sont annoncées pour React, Vue, Angular, Svelte et React Native, et le modèle peut être appelé directement chez son fournisseur. Le langage est annoncé jusqu'à 67 % plus économe en tokens que du JSON, d'après les mesures de l'éditeur. Une passerelle hébergée par Thesys (*Gateway*) et un service de correction (*Autofix*) réparent les réponses invalides ; tous deux exigent une clé de ce service. C1 n'a pas été essayé, son abandon au profit d'OpenUI étant annoncé.

Exemple (test OpenUI). L'essai `experiments/openui` définit un catalogue de six composants bancaires (`Pile` comme racine, `CarteSolde`, `ListeTransactions`, `Transaction`, `ConfirmationVirement`, `Texte`), chacun décrit par un schéma Zod de ses propriétés. La ligne de commande d'OpenUI en tire la consigne du modèle (environ 3 000 caractères) et une spécification JSON. Le serveur appelle Gemini directement, sans la passerelle de Thesys, avec trois outils (solde, transactions, virement), et renvoie le texte en flux ; la page l'affiche au fur et à mesure avec le `Renderer` d'OpenUI. Pour la demande « Montre mes dépenses de plus de 500 euros puis prépare un virement de 50 euros à Paul », le modèle a appelé l'outil des transactions puis répondu :

```
root = Pile([tx_list, confirm])
tx1 = Transaction("Loyer", 780)
tx2 = Transaction("Garage", 620)
tx3 = Transaction("Billets de train", 510)
tx_list = ListeTransactions("Dépenses de plus de 500 €", [tx1, tx2, tx3])
confirm = ConfirmationVirement(50, "Paul")
```

Le bouton « Confirmer » de la carte renvoie un message à l'agent, qui appelle alors l'outil de virement (figure 3). Constats :

— **Réponses valides.** Les cinq réponses obtenues ont été acceptées par le parseur d'OpenUI : aucun composant inconnu, aucune propriété manquante, aucune référence non définie. Le modèle a parfois entouré le code d'un bloc `openui` suivi d'un commentaire, contrairement à la consigne ; ce texte n'a pas été affiché.
— **Consigne coûteuse.** Une réponse sans appel d'outil a consommé 1 071 tokens en entrée, la consigne générée comprise, et les réponses avec outils entre 1 961 et 2 402 (plusieurs appels au modèle). À titre indicatif, le scénario sans outil des mesures de la section 6 en consommait environ 140, avec un autre modèle.
— **Confirmation par consigne seulement.** Rien n'empêche techniquement le modèle d'appeler l'outil de virement sans attendre le clic : la règle figure dans la consigne, sans le blocage qu'offre l'approbation de l'AI SDK ou d'assistant-ui. Les boutons restent aussi actifs après usage.
— **Poids.** La page pèse 127 ko de JavaScript compressé, React compris.

L'essai a utilisé `gemini-3.8-flash`, puis `gemini-3.5-flash` une fois le quota gratuit du premier atteint (20 requêtes par jour).



FIGURE 3 – Essai OpenUI : liste des dépenses et carte de virement générées en OpenUI Lang, puis confirmation par l'utilisateur.

Source : `https://docs.thesys.dev/`

Source : `https://www.openui.com/docs`

5.6 Synthèse

Le tableau 6 résume les frameworks étudiés. CopilotKit a été essayé dans l'exemple d'architecture (section 8.2), l'AI SDK et assistant-ui avec un même agent, OpenUI avec les mêmes outils ; Tambo a été préparé sans être exécuté.

Framework	Licence	Approche	Maturité	Particularité	Essai
CopilotKit	MIT	Statique, déclarative, ouverte	38,1k étoiles ; 0,48 M	Fondé sur AG-UI, rendu A2UI intégré	Exécuté
Vercel AI SDK	Apache 2.0	Statique	27,4k ; 10,1 M	Gère l'état, pas le rendu ; inclut l'appel des modèles	Exécuté
assistant-ui	MIT	Statique, déclarative	12,6k ; 2,0 M	Composants de conversation à assembler, indépendants de l'agent	Exécuté
Tambo	MIT	Statique	11,2k ; 0,01 M	Composants enregistrés par schéma, agent inclus	Compilé
Thesys C1 / OpenUI	Propriétaire / MIT	Déclarative	OpenUI : 11,4k ; 0,18 M	C1 archivé ; OpenUI : langage d'interface compact, catalogue de composants	OpenUI exécuté

TABLE 6 – Frameworks d'interfaces génératives étudiés. Maturité : étoiles GitHub (octobre 2026) ; téléchargements npm hebdomadaires du paquet React (`@copilotkit/react-core`, `@ai-sdk/react`, `@assistant-ui/react`, `@tambo-ai/react`, `@openuidev/react-lang`), semaine du 5 au 11 octobre 2026.

6 Frameworks d'orchestration

6.1 Indépendants

6.1.1 LangChain, LangGraph, LangSmith

LangChain est un framework d'orchestration open source. Il fournit tout ce qui entoure le LLM pour en faire un véritable agent : la consigne, les outils et des modules intermédiaires (middlewares) qui encadrent son comportement. Il est rapide à prendre en main, puisqu'un agent fonctionnel se crée en quelques lignes, la boucle d'appel du modèle et des outils étant gérée par le framework. Il est indépendant du fournisseur de modèle : le même code fonctionne avec GPT, Claude, Mistral ou Gemini.

LangChain repose sur LangGraph, un moteur d'exécution de plus bas niveau développé par le même éditeur. LangGraph représente l'agent sous la forme d'un graphe d'étapes dont l'état est sauvegardé, ce qui permet de reprendre une exécution interrompue et de la mettre en pause pour une validation humaine. Il peut être utilisé directement lorsque l'on souhaite maîtriser précisément le déroulement, par exemple pour imposer une étape de confirmation avant un virement.

LangSmith est la plateforme associée, destinée à tester, déployer et superviser les agents. Elle enregistre le détail de chaque exécution : étapes suivies, outils appelés, paramètres transmis et résultats obtenus. Elle n'est pas réservée à LangChain et peut suivre des agents construits avec d'autres frameworks. Contrairement à LangChain et LangGraph, il s'agit d'un service commercial hébergé : les traces sont transmises par défaut à l'éditeur, qui propose aussi un

hébergement dans l'Union européenne ou chez le client, ce qui constitue un point d'attention dans un contexte bancaire.

Brique	Rôle
LangChain	Framework d'agents : un agent prêt à l'emploi en quelques lignes, avec ses outils et ses middlewares.
LangGraph	Moteur d'exécution : graphe d'étapes, état sauvegardé, pause et reprise.
Deep Agents	Agent préconfiguré pour les tâches longues, construit sur LangChain.
LangSmith	Plateforme de test, de déploiement et de supervision des agents.

LangChain et LangGraph existent en Python et en TypeScript, avec la même API (`create_agent` devient `createAgent`, les outils sont déclarés avec un schéma Zod). Les deux versions n'ont toutefois pas la même audience : le dépôt Python compte environ 147,6k étoiles, le dépôt TypeScript (`langchainjs`) environ 18,3k. L'agent des essais pratiques a été écrit dans les deux langages (`experiments/langchain` et `experiments/langchain-js`).

Source : `https://docs.langchain.com/`

6.1.2 Mastra

Mastra est un framework open source, écrit en TypeScript, qui sert à construire des agents et des workflows IA. Il relie un modèle de langage à des outils et à des données, donne une mémoire aux agents et permet à un humain d'intervenir lorsqu'une tâche demande un jugement.

Mastra réunit dans un même framework les agents, pour lesquels le modèle décide des outils à appeler, et les workflows, qui enchaînent des étapes typées dans un ordre fixé à l'avance. Il fournit également une mémoire de conversation, des évaluations, le traçage des exécutions et un serveur qui expose agents et workflows sous forme d'API. Il est indépendant du fournisseur de modèle et donne accès à plus de 90 fournisseurs par une même interface, dont OpenAI, Anthropic et Google.

Brique	Rôle
Agents	Un modèle, des instructions et des outils. C'est le modèle qui décide quels outils appeler.
Outils	Fonctions que l'agent peut demander, déclarées avec un schéma de leurs paramètres.
Workflows	Suite d'étapes typées, avec branchements et relances, pour un déroulement fixé à l'avance.
Mémoire	Historique des conversations et rappel sémantique.
Évaluations	Notation des réponses par du code ou par un modèle juge.
Traçage et métriques	Suivi des appels de modèles, d'outils et d'étapes, avec latence, tokens et coût.
Serveur	Expose agents et workflows sous forme d'API.
Studio	Interface locale pour tester ses agents.

Chaque outil est déclaré avec un schéma de ses paramètres, écrit avec la bibliothèque Zod : les valeurs produites par le modèle sont validées avant l'exécution. Un outil peut aussi exiger une

approbation humaine, auquel cas l'appel est mis en pause jusqu'à ce que l'application l'accepte ou le refuse. Les workflows peuvent de la même façon se suspendre puis reprendre en conservant leur état, ce qui convient à une opération sensible comme un virement.

Le cœur du framework est distribué sous licence Apache 2.0, tandis que les fonctionnalités destinées aux entreprises relèvent d'une licence propre à l'éditeur.

Source : `https://mastra.ai/docs`

6.1.3 CrewAI

CrewAI est un framework open source, écrit en Python, destiné à la construction de systèmes multi-agents. Il permet de définir des agents, de les regrouper en équipes et d'automatiser des enchaînements de tâches.

CrewAI propose deux modes d'organisation complémentaires. Les *Crews* sont des équipes d'agents autonomes qui collaborent selon des rôles, suivant un processus séquentiel ou hiérarchique. Les *Flows* sont des workflows événementiels qui donnent un contrôle précis sur l'état, les branchements et la logique métier, et qui peuvent reprendre une exécution longue. Les deux peuvent être combinés.

Brique	**Rôle**
Agents	Dotés d'outils, de mémoire, de connaissances et de sorties structurées.
Tâches	Unités de travail confiées aux agents, avec garde-fous et validation humaine possibles.
Crews	Équipes d'agents autonomes qui collaborent selon des rôles.
Flows	Workflows événementiels avec état, branchements et reprise.
Mémoire et connaissances	Contexte conservé et sources de données mises à disposition des agents.
Observabilité	Suivi intégré des exécutions.

Une tâche peut exiger une validation humaine : elle se met alors en pause jusqu'à ce qu'un humain approuve le résultat ou renvoie un commentaire, auquel cas la tâche est relancée avec ce retour. Le framework est distribué sous licence MIT. L'éditeur commercialise en complément une plateforme, CrewAI AMP, pour le déploiement, la supervision et la gouvernance.

Source : `https://docs.crewai.com/`

6.1.4 Pydantic AI

Pydantic AI est un framework open source, écrit en Python, pour construire des agents. Il est développé par l'équipe de Pydantic, la bibliothèque de validation de données utilisée par de nombreux SDK d'IA. Il se présente comme un SDK typé et indépendant du fournisseur de modèle : OpenAI, Anthropic, Google, Mistral ou des modèles locaux sont accessibles en changeant un simple identifiant.

Sa particularité est la place donnée à la validation. Les arguments d'un outil sont vérifiés avant son exécution, et la réponse finale doit respecter le type déclaré par le développeur : si la validation échoue, le modèle est invité à réessayer. Une approbation humaine peut être exigée avant l'exécution d'un outil.

Brique	Rôle
Agent	Boucle d'appel du modèle et des outils, lancée par un simple appel de fonction.
Outils	Fonctions Python dont les arguments sont validés avant exécution.
Sortie structurée	Résultat garanti conforme au type déclaré par le développeur.
Dépendances	Données et services transmis de façon typée aux outils.
Graphes	Paquet séparé pour décrire un déroulement imposé.
Évaluations et observabilité	Tests du comportement des agents et traces au format OpenTelemetry.

Pydantic AI prend en charge les protocoles MCP et AG-UI. Il est distribué sous licence MIT.

Source : `https://pydantic.dev/docs/ai/overview/`

6.2 Liés à un fournisseur de modèles

6.2.1 OpenAI Agents SDK

L'OpenAI Agents SDK est un kit de développement open source, léger, destiné à la construction d'applications agentiques. Il succède à Swarm, le projet expérimental d'OpenAI, et s'utilise d'abord en Python. Il repose sur un petit nombre de primitives : des agents, des passages de relais entre agents, des garde-fous et un traçage intégré.

Le SDK est conçu pour les modèles d'OpenAI, qu'il appelle par défaut. D'autres fournisseurs restent accessibles par l'intermédiaire d'adaptateurs tiers. Il prend en charge le protocole MCP et prévoit des mécanismes pour faire intervenir un humain pendant l'exécution. Il est distribué sous licence MIT.

Brique	Rôle
Agents	Modèles dotés d'instructions et d'outils.
Passages de relais	Délégation d'une tâche à un autre agent spécialisé.
Garde-fous	Validation des entrées et des sorties, exécutée en parallèle.
Sessions	Mémoire persistante du contexte de travail.
Traçage	Suivi intégré pour visualiser et déboguer les exécutions.

Source : `https://openai.github.io/openai-agents-python/`

6.2.2 Claude Agent SDK

Le Claude Agent SDK est la bibliothèque d'Anthropic qui met à disposition des développeurs le moteur de Claude Code : la même boucle d'agent, les mêmes outils et la même gestion du contexte, programmables en Python et en TypeScript. Il s'adresse d'abord aux agents qui agissent sur un environnement de travail, en lisant des fichiers, en exécutant des commandes ou en effectuant des recherches.

Il fonctionne avec les modèles Claude. Un système de permissions détermine quels outils s'exécutent automatiquement et lesquels exigent une approbation.

Contrairement aux autres frameworks présentés, son usage est régi par les conditions commerciales d'Anthropic et non par une licence open source.

Brique	Rôle
Outils intégrés	Lecture et modification de fichiers, exécution de commandes, recherche web.
Permissions	Définition des outils autorisés automatiquement ou soumis à approbation.
Hooks	Code personnalisé exécuté à des moments clés du cycle de l'agent.
Sous-agents	Agents spécialisés lancés pour des sous-tâches.
Sessions et MCP	Contexte conservé entre les échanges et connexion à des outils externes.

Source : `https://code.claude.com/docs/en/agent-sdk/overview`

6.3 Liés à un fournisseur cloud

6.3.1 Google ADK

L'Agent Development Kit (ADK) est le framework open source de Google pour créer, déboguer et déployer des agents. Il est disponible en Python, TypeScript, Go, Java et Kotlin. Il distingue les agents pilotés par un modèle et les agents de workflow, dont le déroulement est séquentiel, parallèle ou en boucle, et propose depuis sa version 2.0 des workflows en graphe aux chemins d'exécution explicites.

ADK est pensé pour Gemini et Google Cloud, mais fonctionne aussi avec d'autres modèles, dont Claude, les modèles d'OpenAI ou des modèles locaux. Il prend en charge les protocoles MCP et A2A, et permet d'exiger une confirmation humaine avant l'exécution d'une action. Il est distribué sous licence Apache 2.0.

Brique	Rôle
Agents LLM	Agents pilotés par un modèle, avec instructions et outils.
Agents de workflow	Déroulement séquentiel, parallèle, en boucle ou en graphe.
Outils	Fonctions, serveurs MCP et API décrites en OpenAPI, avec confirmation des actions.
Sessions et mémoire	Gestion de la conversation, de l'état et des événements.
Évaluation	Critères, simulation d'utilisateur et métriques personnalisées.

Source : `https://adk.dev/`

6.3.2 Microsoft Agent Framework

Microsoft Agent Framework est le successeur direct d'AutoGen et de Semantic Kernel, développé par les mêmes équipes. Il réunit les abstractions simples du premier et les fonctions d'entreprise du second, comme la gestion d'état par session, le typage, les middlewares et la télémétrie. Il est disponible en .NET et en Python, ainsi qu'en Go en préversion.

Il y ajoute des workflows, fonctionnels ou en graphe, qui donnent un contrôle explicite sur l'ordre d'exécution, et une gestion d'état adaptée aux scénarios longs ou avec validation humaine. Il s'intègre à Microsoft Foundry et à Azure OpenAI, et prend aussi en charge OpenAI, Anthropic et Ollama. Il est distribué sous licence MIT.

Brique	**Rôle**
Agents	Appel du modèle, des outils et des serveurs MCP.
Workflows	Chemins d'exécution explicites reliant agents et fonctions.
Sessions	Gestion de l'état des conversations.
Middlewares	Interception des actions de l'agent.
Intégrations	Fournisseurs de modèles, outils, évaluation et interfaces.

Source : `https://learn.microsoft.com/en-us/agent-framework/overview/`

6.3.3 AWS Strands Agents

Strands Agents est une boîte à outils open source, issue d'AWS, pour construire des agents destinés à la production. Elle est disponible en Python et en TypeScript. Elle fournit une boucle d'agent, des outils intégrés et plusieurs schémas multi-agents : agents utilisés comme outils, essaim, graphe et workflow.

Strands s'intègre à Amazon Bedrock et aux services de déploiement d'AWS, tout en prenant en charge d'autres fournisseurs de modèles, dont Anthropic, OpenAI, Google, Mistral et Ollama. Il prend en charge les protocoles MCP et A2A, ainsi que les interruptions pour validation humaine. Il est distribué sous licence Apache 2.0.

Brique	**Rôle**
Boucle d'agent	Appel du modèle et des outils jusqu'à la fin de la tâche.
Outils	Outils intégrés et outils MCP.
Multi-agents	Agents comme outils, essaim, graphe et workflow.
Sessions et hooks	État conservé et points d'intervention dans le cycle de l'agent.
Observabilité	Métriques, traces et journaux.

Source : `https://strandsagents.com/`

6.4 Écosystème Java

6.4.1 Spring AI

Spring AI est le projet de l'écosystème Spring, en Java, pour intégrer l'intelligence artificielle dans les applications d'entreprise. Il s'inspire de projets Python comme LangChain sans en être un portage. Il fournit des abstractions communes à de nombreux fournisseurs de modèles, ce qui permet d'en changer avec peu de modifications de code.

Plus qu'un framework d'agents, Spring AI est une couche d'intégration : il couvre l'appel d'outils, la mémoire de conversation, la sortie structurée et la recherche documentaire. Il prend en charge le protocole MCP. Sa version stable actuelle est la 2.0, distribuée sous licence Apache 2.0.

Brique	Rôle
ChatClient	API d'échange avec les modèles de conversation.
Appel d'outils	Exécution de fonctions demandées par le modèle.
Advisors	Traitements réutilisables appliqués aux échanges avec le modèle.
Mémoire et RAG	Historique des conversations et recherche dans des bases vectorielles.
Observabilité et évaluation	Suivi des appels et contrôle des réponses générées.

Source : `https://docs.spring.io/spring-ai/reference/`

6.5 Synthèse et comparaison

6.5.1 Caractéristiques générales

Les dix frameworks étudiés remplissent le même rôle : ils gèrent la boucle entre le modèle et les outils, et placent l'exécution des actions sous le contrôle de l'application. Ils se distinguent par leur langage, par leur degré de dépendance à un fournisseur et par la manière dont ils encadrent le déroulement d'une tâche. Le tableau 7 en résume les caractéristiques générales.

Framework	Langage	Licence	Particularité
LangChain / LangGraph	Python, TypeScript	MIT	Écosystème le plus répandu, déroulement en graphe
Mastra	TypeScript	Apache 2.0	Agents et workflows dans un même framework
CrewAI	Python	MIT	Équipes d'agents organisées par rôles
Pydantic AI	Python	MIT	Validation stricte des entrées et des sorties
OpenAI Agents SDK	Python	MIT	Léger, centré sur les modèles d'OpenAI
Claude Agent SDK	Python, TypeScript	Conditions commerciales	Moteur de Claude Code, centré sur les modèles Claude
Google ADK	Python, TypeScript, Go, Java, Kotlin	Apache 2.0	Lié à Gemini et Google Cloud, protocoles A2A et MCP
Microsoft Agent Framework	.NET, Python	MIT	Successeur d'AutoGen et de Semantic Kernel
AWS Strands Agents	Python, TypeScript	Apache 2.0	Lié à Amazon Bedrock, schémas multi-agents
Spring AI	Java	Apache 2.0	Couche d'intégration de l'écosystème Spring

TABLE 7 – Caractéristiques générales des frameworks d'orchestration étudiés.

6.5.2 Protocole des essais pratiques

Des essais pratiques sont réalisés dans le dossier `experiments/` du dépôt. Le même agent est reproduit à l'identique d'un framework à l'autre :

— **Modèle :** Gemini (`gemini-3.5-flash-lite`), avec un délai maximal de 60 secondes par appel, sans nouvelle tentative automatique lorsque le framework permet de la désactiver.
— **Consigne :** « Tu es un assistant bancaire. Utilise les outils, n'invente jamais de données. »
— **Outils :** deux outils, qui s'appuient sur des données fictives :
  — `get_balance(account)` renvoie le solde d'un compte ;
  — `get_transactions(min_amount)` renvoie les transactions supérieures à un montant.

Quatre scénarios sont posés à cet agent (tableau 8). Chaque scénario est répété 10 fois, avec 10 secondes de pause entre deux essais afin de respecter le quota gratuit du modèle. Un essai est compté comme réussi lorsque l'ensemble des outils appelés correspond exactement à l'ensemble attendu.

N°	Scénario	Question posée	Outils attendus
1	Solde du compte courant	Quel est le solde de mon compte courant ?	`get_balance`
2	Dépenses supérieures à 500 €	Quelles dépenses dépassent 500 € ?	`get_transactions`
3	Solde et dépenses	Quel est mon solde courant, et quelles dépenses dépassent 500 € ?	`get_balance`, `get_transactions`
4	Salutation, sans outil	Bonjour, que peux-tu faire pour moi ?	aucun

TABLE 8 – Scénarios des essais pratiques.

Le premier scénario teste l'appel d'un seul outil, le deuxième l'extraction d'un paramètre numérique à transmettre à l'outil, le troisième la combinaison des deux outils dans une même demande, et le dernier la capacité de l'agent à ne pas appeler d'outil lorsque la question ne le demande pas.

Pour chaque scénario sont relevés la durée de réponse (médiane, minimum et maximum), le nombre d'appels au modèle, les tokens en entrée et en sortie (médianes, tels que rapportés par chaque framework) et le taux de réussite. Chaque script `measure` affiche ces valeurs au format CSV ; elles sont conservées dans un fichier `resultats.csv` pour tous les frameworks sauf LangChain (Python) et Mastra, dont la sortie n'a pas été enregistrée. À titre d'illustration, l'agent se déclare ainsi avec LangChain :

```python
def get_balance(account: str) -> str:
    """Renvoie le solde d'un compte ('courant' ou 'livret')."""
    return {"courant": "1 250 €", "livret": "8 400 €"}.get(account, "Compte
        inconnu")

model = init_chat_model("google_genai:gemini-3.5-flash-lite", timeout=60,
    max_retries=0)
agent = create_agent(
    model=model,
    tools=[get_balance, get_transactions],
    system_prompt="Tu es un assistant bancaire. Utilise les outils, n'invente
        jamais de données.",
)
```

Le même agent s'écrit en TypeScript avec LangChain.js :

```typescript
const getBalance = tool(
  async ({ account }) => soldes[account] ?? "Compte inconnu",
  { name: "get_balance", description: "Renvoie le solde d'un compte ('courant'
      ou 'livret')."},
```

```
    schema: z.object({ account: z.string() }) },
);
const agent = createAgent({
  model: new ChatGoogleGenerativeAI({ model: "gemini-3.5-flash-lite",
      maxRetries: 0 }),
  tools: [getBalance, getTransactions],
  systemPrompt: "Tu es un assistant bancaire. Utilise les outils, n'invente
      jamais de données.",
});
```

Conditions d'exécution. Python 3.14 (3.13 pour CrewAI, qui ne s'installe pas en 3.14), Node.js 26, octobre 2026. OpenAI Agents SDK appelle Gemini par son point d'accès compatible avec l'API d'OpenAI ; tous les autres frameworks utilisent leur connecteur Gemini. Le Claude Agent SDK, limité aux modèles Claude, et Spring AI, faute de JDK sur le poste, ont été préparés sans être exécutés (`experiments/claude-agent-sdk`, `experiments/spring-ai`).

6.5.3 Résultats

Le tableau 9 présente les mesures obtenues. Les mesures de CrewAI sont réalisées avec les traces désactivées. L'AI SDK de Vercel, présenté à la section 5, a été mesuré avec le même protocole à titre de comparaison.

	Durée médiane (s)				**Tokens en entrée**				
Framework	**1**	**2**	**3**	**4**	**1**	**2**	**3**	**4**	**Réussite**
LangChain (Python)	1,06	1,14	1,26	0,77	320	344	392	142	40/40
LangChain.js	1,00	1,18	1,32	0,77	322	346	452	143	40/40
Mastra	1,09	1,13	1,37	0,84	328	352	408	142	40/40
CrewAI	1,15	1,16	1,25	0,80	400	422	472	182	40/40
Pydantic AI	1,23	1,34	1,41	0,78	324	348	398	143	39/39[a]
OpenAI Agents SDK	1,08	1,14	1,28	0,75	388	412	461	176	40/40
Google ADK	1,35	1,27	1,22	0,79	418	442	490	191	40/40
Microsoft Agent Framework	1,13	1,08	1,26	0,75	388	412	460	176	40/40
AWS Strands Agents	1,15	1,58	1,49	0,86	357	381	432	159	40/40
Vercel AI SDK	1,03	1,15	1,29	0,72	328	352	408	142	36/36[b]

TABLE 9 – Résultats des essais pratiques (10 répétitions par scénario ; scénarios numérotés comme au tableau 8). [a] Un essai perdu sur une erreur de quota (HTTP 429), non compté. [b] Quatre essais du scénario 4 perdus sur une erreur de quota journalier (HTTP 429), non comptés.

Le nombre d'appels au modèle est le même pour tous les frameworks : deux lorsqu'un outil est utilisé (la demande d'outil, puis la réponse finale), un seul sinon. Dans le troisième scénario, les deux outils sont demandés dans un même appel. Les tokens en sortie sont eux aussi très proches (33 pour le premier scénario, 99 environ pour le troisième, 52 à 68 pour le dernier). Les valeurs détaillées (minimum, maximum, tokens en sortie) figurent dans les fichiers `resultats.csv` lorsqu'ils existent.

Plusieurs constats se dégagent de ces mesures :

— **Fiabilité.** Tous les frameworks obtiennent la totalité des réussites : le choix des outils et

l'extraction du paramètre dépendent du modèle, identique dans tous les essais, et non du framework.

— **Durée.** Les durées médianes restent comprises entre 0,72 et 1,6 s selon le scénario. Sur un même scénario, l'écart entre le framework le plus rapide et le plus lent atteint au plus 0,5 s (scénario 2, Strands), et 0,1 à 0,35 s sur les autres scénarios ; l'essentiel du temps est passé dans l'appel au modèle. Les valeurs maximales isolées (6,73 s pour LangChain, 5,65 s pour LangChain.js, 3,55 s pour CrewAI et OpenAI Agents SDK) relèvent de la variabilité du service distant.

— **Python et TypeScript.** LangChain et LangChain.js donnent des durées et des tokens quasiment identiques sur trois scénarios ; sur le troisième, la version TypeScript consomme 60 tokens de plus en entrée (452 contre 392), sans que la cause ait été isolée.

— **Tokens fixes.** La différence la plus nette porte sur les tokens en entrée sans outil appelé (scénario 4), qui mesurent ce que le framework ajoute à la question : 142 à 143 pour LangChain, Mastra, Pydantic AI et l'AI SDK, 159 pour Strands, 176 pour OpenAI Agents SDK et Microsoft Agent Framework, 182 pour CrewAI (qui ajoute le rôle, l'objectif et le contexte de l'agent) et 191 pour Google ADK. Ce surcoût se retrouve à chaque appel au modèle.

— **Traces.** Avec CrewAI, l'activation des traces porte la durée de réponse à 2,4 – 2,8 s, contre 0,8 – 1,3 s lorsqu'elles sont désactivées.

6.5.4 Comparaison selon les critères retenus

Le tableau 10 présente les critères retenus pour la comparaison. La maturité est approchée par la popularité du dépôt GitHub de chaque projet, relevée en octobre 2026. La facilité d'intégration et les performances sont évaluées par les essais pratiques décrits ci-dessus ; la colonne sécurité reprend les mécanismes documentés et les constats faits pendant les essais.

Table 10: Comparaison des frameworks selon les critères retenus.

Framework	**Maturité (étoiles)**	**Facilité d'intégration**	**Performances**	**Sécurité**
LangChain / LangGraph (Python)	147,6k	Installation simple. Changement de modèle en une ligne.	0,8 à 1,3 s par réponse, 142 tokens fixes, 40/40 réussites.	Validation humaine par middleware. Faille critique (CVE-2025-68664) corrigée en décembre 2025. Traces LangSmith optionnelles.
LangChain.js	18,3k	Même API qu'en Python ; exécution directe par Node.js 26.	0,8 à 1,3 s par réponse, 143 tokens fixes, 40/40 réussites.	Middlewares de validation humaine et de masquage des données personnelles. Faille similaire (CVE-2025-68665) corrigée.
Mastra	28,7k	Installation simple (TypeScript, Node 22 ou plus).	0,8 à 1,4 s par réponse, 142 tokens fixes, 40/40 réussites.	Approbation par outil et paramètres validés par schéma. Traçage intégré.

Framework	Maturité (étoiles)	Facilité d'intégration	Performances	Sécurité
CrewAI	59,5k	Exige Python 3.13 au plus : installation impossible en 3.14. Sortie console abondante.	0,8 à 1,3 s traces désactivées, 2,4 à 2,8 s activées. 182 tokens fixes, 40/40 réussites.	Télémétrie technique par défaut. Partage des traces proposé au premier lancement.
Pydantic AI	20,5k	Installation simple. Mesures lues directement dans le résultat. Un bandeau invite à activer son service d'observabilité (Logfire) à chaque lancement.	0,8 à 1,4 s par réponse, 143 tokens fixes, 39/39 réussites.	Arguments des outils validés, approbation possible. Aucun envoi de données par défaut.
OpenAI Agents SDK	30,0k	Gemini utilisable seulement par le point d'accès compatible OpenAI.	0,8 à 1,3 s par réponse, 176 tokens fixes, 40/40 réussites.	Traces envoyées à OpenAI par défaut : à désactiver sans clé OpenAI. Garde-fous en entrée et en sortie.
Claude Agent SDK	8,2k	Modèles Claude uniquement ; outils servis par un serveur MCP intégré.	Non mesuré.	Permissions par outil ; outils intégrés (fichiers, commandes) à désactiver explicitement.
Google ADK	21,8k	Connecteur Gemini natif, outils écrits comme de simples fonctions ; gestion explicite des sessions.	0,8 à 1,4 s par réponse, 191 tokens fixes, 40/40 réussites.	Confirmation humaine avant une action.
Microsoft Agent Framework	14,1k	Le paquet `agent-framework` installe une trentaine de sous-paquets ; connecteur Gemini en version bêta.	0,8 à 1,3 s par réponse, 176 tokens fixes, 40/40 réussites.	Middlewares d'interception des actions de l'agent.
AWS Strands Agents	8,8k	Connecteur Gemini en option ; outils et métriques lus directement dans le résultat.	0,9 à 1,6 s par réponse, 159 tokens fixes, 40/40 réussites.	Interruptions pour validation humaine.
Spring AI	9,5k	Code préparé, non compilé (pas de JDK sur le poste).	Non mesuré.	À évaluer.

Les « tokens fixes » désignent les tokens en entrée consommés par un appel sans outil (scénario 4) : ils mesurent ce que le framework ajoute à chaque requête en plus de la question de l'utilisateur.

Source : `https://thehackernews.com/2025/12/critical-langchain-core-vulnerability.html`

7 Observabilité

L'observabilité consiste à pouvoir reconstituer, après coup, ce qu'un agent a fait pour traiter une demande : quels appels au modèle il a effectués et avec quelle consigne, quels outils il a appelés, avec quels paramètres et quels résultats, combien de temps et de tokens chaque étape a coûté, et où une erreur s'est produite. C'est l'équivalent des journaux d'une application classique, adapté à un programme dont le comportement n'est pas déterministe : la même question peut conduire à des appels d'outils différents d'une exécution à l'autre. Elle sert à déboguer, à suivre les coûts et les performances, et à conserver la preuve de ce que l'agent a fait, ce qui répond au pilier « industrialisation et robustesse » du sujet.

7.1 Traces et standards

L'unité d'observation est la *trace*, qui correspond au traitement d'une demande. Elle se compose de *spans*, un par étape, imbriqués les uns dans les autres : l'exécution de l'agent contient les appels au modèle et les appels d'outils, chacun avec ses entrées, ses sorties, sa durée et, pour le modèle, le nombre de tokens.

Ce découpage vient d'OpenTelemetry, le standard ouvert d'observabilité des applications. Des conventions propres à l'IA générative, maintenues dans un dépôt dédié, définissent les spans, métriques et événements des clients de modèles, y compris pour MCP. Les outils présentés ci-dessous acceptent tous des traces OpenTelemetry, ce qui évite de dépendre de l'un d'eux pour l'instrumentation.

Source : `https://github.com/open-telemetry/semantic-conventions-genai`

7.2 Outils

LangSmith. Plateforme de l'éditeur de LangChain (section 6). Elle s'active sur un agent LangChain sans modifier son code. C'est un service hébergé, aux États-Unis ou dans l'Union européenne. L'offre *Developer* est gratuite pour un seul utilisateur, avec 5 000 traces par mois ; l'offre *Plus* coûte 39 dollars par utilisateur et par mois, avec 10 000 traces. Les traces de base sont conservées 14 jours. L'hébergement chez le client n'est proposé qu'avec l'offre *Enterprise*.

Source : `https://www.langchain.com/pricing`

Source : `https://docs.langchain.com/langsmith/self-hosted`

Langfuse. Plateforme open source (licence MIT, à l'exception des fonctions d'entreprise, environ 36k étoiles). Elle s'auto-héberge avec Docker Compose pour un usage local, ou sur Kubernetes en production ; elle s'appuie alors sur PostgreSQL, ClickHouse, Redis et un stockage de fichiers de type S3. Sa documentation indique qu'elle peut fonctionner sur site, sans accès à Internet. L'intégration à LangChain se fait par un *callback*, et elle couvre aussi l'AI SDK de Vercel et Mastra.

Source : `https://github.com/langfuse/langfuse`

Source : `https://langfuse.com/self-hosting`

Arize Phoenix. Outil d'observabilité et d'évaluation, fondé sur OpenTelemetry et sur l'instrumentation OpenInference du même éditeur (environ 11,9k étoiles). Il s'installe par `pip` et se

lance en local par `phoenix serve`. Sa licence, l'Elastic License 2.0, permet l'usage interne mais n'est pas une licence open source au sens strict. Il couvre LangChain et LangGraph, l'AI SDK de Vercel et Mastra.

Source : `https://github.com/Arize-ai/phoenix`

Outil	Licence	Hébergement des traces	Point notable
LangSmith	Commerciale	Éditeur (US ou UE) ; chez le client en offre Enterprise	Intégration native à LangChain ; gratuit pour un utilisateur
Langfuse	MIT (sauf fonctions d'entreprise)	Auto-hébergé ou service de l'éditeur	Auto-hébergement complet, plusieurs services à faire tourner
Phoenix	Elastic License 2.0	Local	Installation la plus légère ; essayé

TABLE 11 – Outils d'observabilité étudiés.

Dans un contexte bancaire, l'emplacement des traces est la question principale : une trace contient la totalité des échanges, donc les questions des clients et les données renvoyées par les outils. Les envoyer à un service tiers revient à lui transmettre ces données, ce qui relève du RGPD et, pour un établissement financier, du suivi des prestataires imposé par DORA.

7.3 Essai : traces conservées en local

L'essai `experiments/observabilite` reprend l'agent LangChain des mesures (section 6) et l'instrumente avec OpenInference. Les traces sont envoyées à Phoenix, lancé sur le poste ; trois lignes suffisent, sans modifier l'agent :

```
fournisseur = register(endpoint="http://localhost:6006/v1/traces",
                       project_name="assistant-bancaire")
LangChainInstrumentor().instrument(tracer_provider=fournisseur)
```

Deux questions ont produit deux traces et seize spans (figure 4). Chaque trace contient l'exécution de l'agent, les appels au modèle avec la consigne, la réponse et le nombre de tokens (de 142 à 388 en entrée selon l'étape), et les appels d'outils avec leurs paramètres (`{"min_amount": 500}`) et leurs résultats. Les traces sont stockées dans une base SQLite du dossier de travail de Phoenix et consultées dans son interface web, sur le poste.

Deux réglages comptent pour que rien ne sorte de la machine. La variable `PHOENIX_TELEMETRY_ENABLED=false`, utilisée pendant l'essai, désactive les statistiques d'usage de l'interface. Phoenix écoute par ailleurs par défaut sur toutes les interfaces réseau ; sa configuration prévoit la variable `PHOENIX_HOST` pour limiter l'accès au poste (non essayée). L'essai a utilisé `gemini-3.5-flash`.

LangSmith et Langfuse n'ont pas été essayés : le premier demande la création d'un compte, le second Docker, absent du poste utilisé.

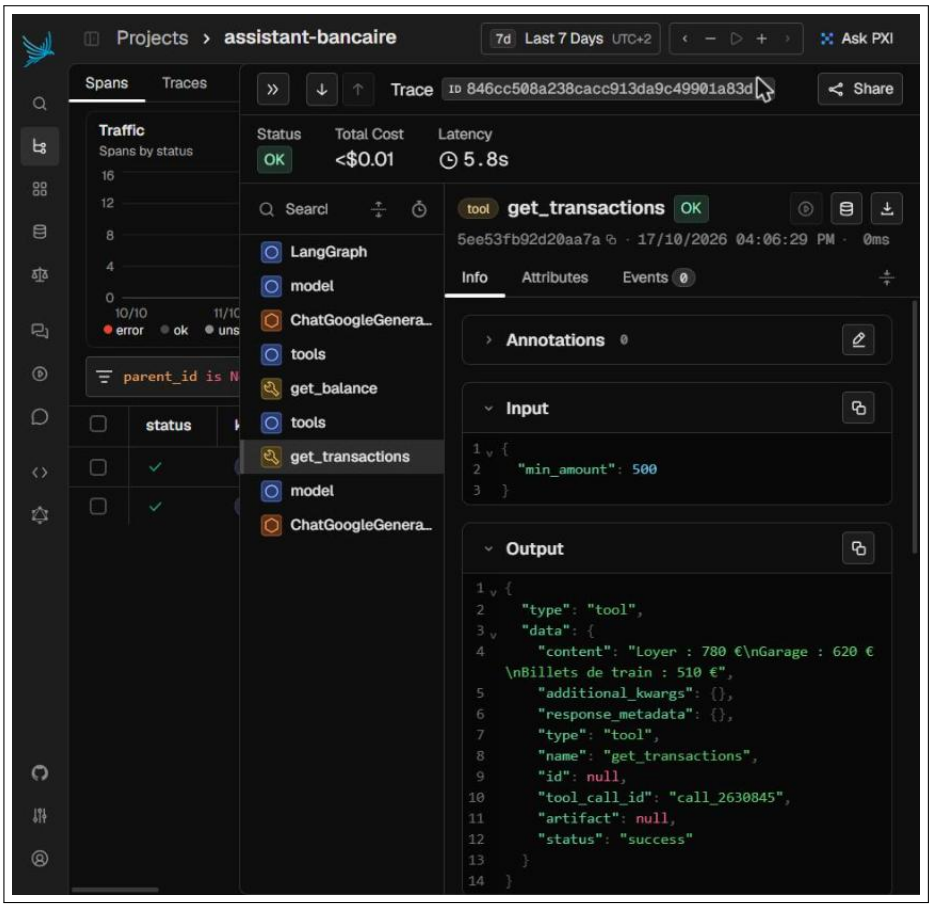


FIGURE 4 – Trace d'une demande dans Phoenix : à gauche les étapes de l'agent, à droite les paramètres et le résultat de l'outil `get_transactions`.

8 Synthèse

8.1 Comment ces technologies collaborent

Les technologies étudiées ne sont pas concurrentes : chacune occupe une frontière différente du système. L'architecture envisagée repose sur un seul agent, placé côté serveur. Le tableau 12 rappelle la question à laquelle chaque technologie répond, et la figure 5 montre comment elles s'assemblent.

Frontière	**Technologie**	**Question à laquelle elle répond**
Agent – modèle	Framework d'orchestration (LangChain)	Qui gère la boucle entre le modèle et les outils ?
Modèle – outils	Function calling	Comment le modèle demande-t-il l'appel d'un outil ?
Agent – serveur d'outils	MCP	Comment l'agent découvre-t-il et appelle-t-il les outils regroupés dans un serveur dédié ?
Agent – agent	A2A	Comment un agent délègue-t-il une tâche à un autre agent ? Inutile tant que l'architecture ne compte qu'un agent, mais prêt si d'autres agents s'ajoutent.
Agent – interface	AG-UI	Comment l'exécution de l'agent est-elle diffusée à la page ?
Contenu de l'interface	A2UI	Comment l'agent décrit-il ce qu'il faut afficher ?
Rendu	Framework d'interface générative (CopilotKit)	Qui transforme cette description en composants à l'écran ?

Table 12 – Rôle de chaque technologie dans l'architecture.

WebMCP n'y figure pas : il s'adresse aux agents intégrés au navigateur (section 2.3), alors que l'agent de l'application s'exécute côté serveur. Il pourra être ajouté plus tard, au-dessus des mêmes outils, si la banque souhaite rendre son site utilisable par ces agents.

Tout le traitement a lieu côté serveur. La page ne fait qu'envoyer la demande de l'utilisateur et afficher ce que l'agent renvoie ; elle n'exécute aucun outil. Les outils bancaires (consulter les comptes, lister les transactions, préparer un virement) sont regroupés dans un serveur MCP, qui appelle l'API bancaire. L'agent n'a donc pas d'accès direct aux données : il passe par ces outils, dont la liste et les paramètres sont fixés par l'application. L'identité de l'utilisateur connecté devra être transmise jusqu'à l'API, pour que chaque appel ne porte que sur ses propres comptes.

Placer les outils dans un serveur MCP plutôt que dans le code de l'agent ne change rien pour un agent unique, mais prépare l'extension à plusieurs agents (cadres en pointillés de la figure 5). L'agent assistant resterait le seul interlocuteur de l'interface et déléguerait, par A2A, les demandes relevant d'un domaine particulier à des agents spécialisés ; ces agents utiliseraient les mêmes outils, par le même serveur MCP.

Le traitement d'une demande telle que « Montre-moi mes comptes » suit alors les étapes suivantes.

1. **Saisie.** L'utilisateur écrit sa demande. Le framework d'interface l'envoie à l'agent en AG-UI, avec l'historique.
2. **Décision.** Le framework d'orchestration transmet au modèle la consigne, la demande et la description des outils, obtenue du serveur MCP. Le modèle ne répond pas par un texte :

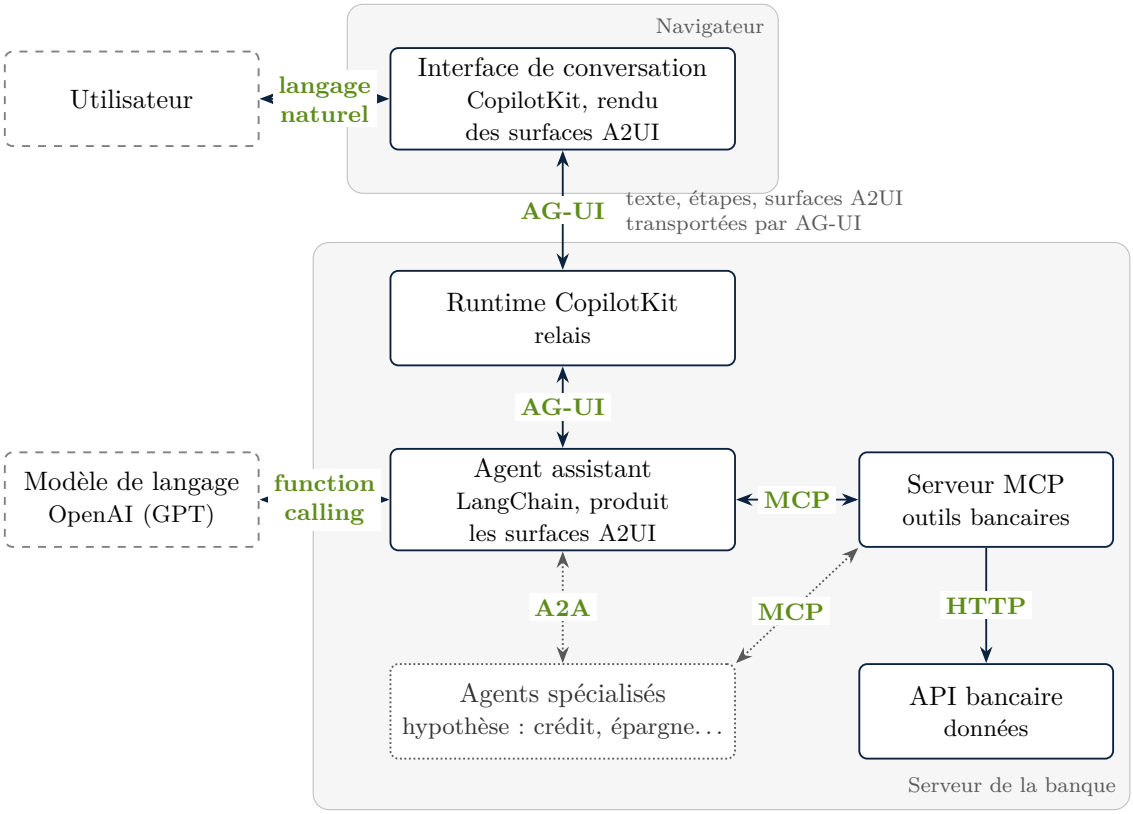


FIGURE 5 – Collaboration des technologies autour de l'agent assistant. Les cadres en tirets sont extérieurs au système développé ; ceux en pointillés représentent une extension possible à plusieurs agents.

il demande l'appel de l'outil `consulter_comptes` (function calling).

3. **Exécution.** L'agent appelle l'outil sur le serveur MCP, qui interroge l'API bancaire et renvoie le résultat. L'appel est signalé à la page par un événement AG-UI.
4. **Description de l'interface.** Le modèle dispose maintenant des données. Il demande leur affichage : l'agent produit la description A2UI d'une carte, c'est-à-dire une liste de composants du catalogue et les données à y placer.
5. **Diffusion.** Chacune de ces étapes est émise au fil de l'eau vers la page sous la forme d'événements AG-UI : début de l'exécution, appels d'outils, résultats, puis texte de la réponse fragment par fragment.
6. **Rendu.** Le framework d'interface affiche le texte dans la conversation et transforme la description A2UI en composants réels. Le modèle n'a produit aucun code exécuté par le navigateur.

Cette répartition a trois conséquences pour le projet. Chaque brique est remplaçable : l'agent peut changer de framework d'orchestration sans que l'interface soit modifiée, puisqu'elle ne connaît que AG-UI. Le modèle n'agit jamais directement : il ne fait que demander des appels d'outils et décrire des interfaces, que le code de l'application exécute et affiche. Enfin, les outils sont définis une seule fois, dans le serveur MCP, et restent utilisables par tout agent ajouté par la suite.

8.2 Exemple d'architecture

Un exemple minimal est fourni dans le dossier `experiments/architecture` du dépôt. Il se compose de trois processus (tableau 13) et reprend les données fictives des essais précédents.

Il a été réalisé avant le choix de placer les outils côté serveur et en diffère sur ce point : ses outils sont définis dans la page, déclarés avec WebMCP et transmis à l'agent par AG-UI. Il valide l'assemblage de LangChain, AG-UI, A2UI et CopilotKit ; l'accès aux outils par un serveur MCP a été essayé séparément (section 2.2).

Processus	Port	Rôle et technologies
`agents/assistant.py`	8000	Agent unique : LangChain et Gemini, exposé en AG-UI, avec un outil d'affichage A2UI. Sert aussi l'API bancaire.
`frontend/runtime.ts`	4000	Runtime CopilotKit, relais entre la page et l'agent.
`frontend/src/`	5173	Page React : conversation CopilotKit et outils de la page.

Table 13 – Processus de l'exemple d'architecture.

Définir les outils dans la page. Chaque outil est défini une seule fois, avec le schéma de ses paramètres. Il ne fait qu'appeler l'API bancaire.

```
// outils.ts
export const consulterTransactions = {
  name: "consulter_transactions",
  description: "Renvoie les transactions des trois derniers mois " +
               "supérieures à un montant en euros.",
  parameters: z.object({ min_amount: z.number() }),
  handler: async ({ min_amount }) =>
    (await fetch(`${API}/transactions?min_amount=${min_amount}`)).json(),
};
```

Les déclarer avec WebMCP et les transmettre à l'agent. La même définition sert deux fois : elle est déclarée au navigateur, lorsque celui-ci le permet, et transmise à l'agent par le framework d'interface.

```
// webmcp.ts : déclaration au navigateur
const contexte = document.modelContext;
if (!contexte) return; // navigateur sans WebMCP
for (const outil of OUTILS) {
  contexte.registerTool({
    name: outil.name,
    description: outil.description,
    inputSchema: z.toJSONSchema(outil.parameters),
    execute: async (parametres) => JSON.stringify(await outil.handler(
        parametres)),
  });
}

// App.tsx : transmission à l'agent, exécution dans la page
useFrontendTool(consulterComptes);
useFrontendTool(consulterTransactions);
```

Orchestrer avec LangChain et diffuser en AG-UI. L'agent est déclaré comme dans les essais précédents, à une différence près : il ne définit aucun outil métier. Un middleware lui ajoute, à chaque requête, les outils reçus de la page. Une seule ligne suffit ensuite à l'exposer en AG-UI : l'adaptateur traduit chaque étape de l'agent en événements.

```
agent = create_agent(
    model=init_chat_model("google_genai:gemini-3.5-flash-lite", timeout=60),
    tools=[afficher_carte],
    middleware=[CopilotKitMiddleware()],
    system_prompt="Tu es un assistant bancaire. Utilise les outils, "
                  "n'invente jamais de données.",
    checkpointer=InMemorySaver(),
)

app = FastAPI()
add_langgraph_fastapi_endpoint(app, LangGraphAgent(name="assistant", graph=
    agent), "/")
```

Décrire une interface en A2UI. Le seul outil de l'agent sert à l'affichage. Il ne renvoie pas de texte, mais trois opérations A2UI : la création d'une surface, la liste de ses composants et les données à afficher. Les composants ne contiennent pas les valeurs : ils y font référence par un chemin dans le modèle de données, et la ligne sert de gabarit, répété pour chaque élément de la liste.

```
CARTE = [
    {"id": "root", "component": "Card", "child": "contenu"},
    {"id": "contenu", "component": "Column", "children": ["titre", "liste"]},
    {"id": "titre", "component": "Text", "text": {"path": "/titre"}, "variant"
        : "h3"},
    {"id": "liste", "component": "List",
     "children": {"path": "/lignes", "componentId": "ligne"}},
    {"id": "ligne", "component": "Row", "children": ["libelle", "valeur"],
     "justify": "spaceBetween"},
    {"id": "libelle", "component": "Text", "text": {"path": "libelle"}},
    {"id": "valeur", "component": "Text", "text": {"path": "valeur"}, "variant
```

```
        ": "h4"},
]

def afficher_carte(titre: str, lignes: list[Ligne]) -> str:
    """Affiche à l'utilisateur une carte : un titre et des lignes."""
    surface = f"carte-{uuid4().hex[:8]}"
    return wrap_as_operations_envelope([
        create_surface(surface, BASIC_CATALOG_ID),
        update_components(surface, CARTE),
        update_data_model(surface, {"titre": titre, "lignes": lignes}),
    ])
```

Afficher avec CopilotKit. Côté serveur, le runtime déclare l'agent par son adresse AG-UI et active le traitement de l'A2UI. Côté page, un seul composant affiche la conversation, les appels d'outils et les surfaces.

```
// runtime.ts
const runtime = new CopilotRuntime({
  agents: { default: new HttpAgent({ url: "http://127.0.0.1:8000/" }) },
  a2ui: {},
});
app.use(createCopilotExpressHandler({ runtime, basePath: "/api/copilotkit" }))
    ;

// main.tsx
<CopilotKitProvider runtimeUrl="http://localhost:4000/api/copilotkit">
  <CopilotChat />
</CopilotKitProvider>
```

Résultat. La figure 6 montre la réponse obtenue. L'agent a d'abord appelé l'outil de consultation des comptes, exécuté dans la page, puis a demandé l'affichage du résultat : la carte est la surface A2UI rendue par CopilotKit.

Limites de l'exemple.

— La déclaration des outils au navigateur n'a pu être vérifiée qu'ensuite, une fois l'API WebMCP activée dans Chrome (section 2.3). Pendant l'essai de l'exemple, seule la transmission des outils à l'agent et leur exécution dans la page ont été vérifiées.
— La structure de la carte A2UI est écrite à l'avance : le modèle en choisit le titre et les lignes, mais ne compose pas l'interface. La génération complète par le modèle, proposée par l'adaptateur AG-UI de LangChain, a échoué avec Gemini sur une erreur interne de la bibliothèque ; elle reste à évaluer.
— Sur une demande combinant comptes et dépenses, une seule des deux cartes attendues a été affichée.
— Les réponses ont été lentes, plusieurs dizaines de secondes par demande. L'exécution d'un outil dans la page impose un aller-retour et un appel au modèle supplémentaires, mais la cause n'a pas été isolée.
— Aucune validation humaine n'est mise en place, l'exemple ne comportant pas d'action sensible.

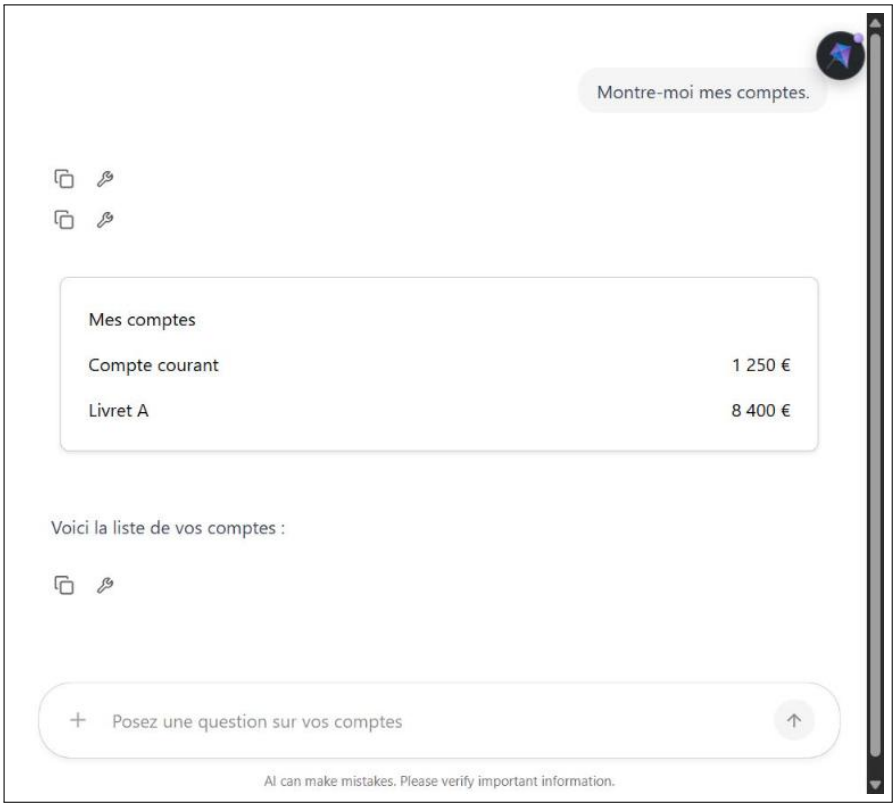


FIGURE 6 – Réponse de l'exemple d'architecture à la demande « Montre-moi mes comptes ».

9 Choix technologiques

Choix retenus par l'équipe. Les technologies ci-dessous sont celles retenues à l'issue de l'état de l'art et des essais.

9.1 Ce que les essais permettent de conclure

- **Le framework d'orchestration n'est pas discriminant sur les performances.** Avec le même modèle, tous les frameworks mesurés réussissent tous les scénarios, avec des durées médianes de 0,72 à 1,6 s et des écarts entre frameworks inférieurs à la variabilité du modèle (tableau 9). Seul le surcoût en tokens les distingue nettement (de 142 à 191 tokens fixes par appel).
- **Les protocoles d'interface sont complémentaires.** AG-UI transporte l'exécution de l'agent, A2UI décrit l'interface et CopilotKit l'affiche ; l'exemple d'architecture (section 8.2) montre qu'ils s'assemblent autour d'un agent LangChain.
- **Les bibliothèques d'interface couvrent le même besoin de base.** CopilotKit, l'AI SDK, assistant-ui et OpenUI ont tous permis d'afficher des composants bancaires à partir des réponses de l'agent ; les trois derniers ont aussi permis de faire confirmer un virement par l'utilisateur, l'exemple CopilotKit ne comportant pas d'action sensible. Ils diffèrent par le protocole utilisé, la prise en charge d'A2UI et le poids de la page.
- **Les choix se font donc sur d'autres critères** : intégration entre les briques, maturité de l'écosystème, maîtrise des données et possibilité d'évoluer (plusieurs agents, autre modèle).

Les sections suivantes reprennent les briques dans l'ordre du document. Le tableau 14 les résume.

Brique	Choix	Non retenu
Description de l'interface	A2UI	OpenUI, HTML libre (MCP Apps)
Agent – interface	AG-UI	Flux propre à l'AI SDK
Agent – outils	Function calling, outils dans un serveur MCP	WebMCP, UTCP
Agent – agent	A2A, si plusieurs agents	–
Framework front-end	React, avec Vite	Vue, Angular, Svelte ; Next.js
Interface générative	CopilotKit	AI SDK UI, assistant-ui, Tambo
Orchestration	LangChain, en Python	LangChain.js ; Mastra, CrewAI, Pydantic AI…
Observabilité	LangSmith (offre gratuite)	Langfuse, Phoenix (pour un usage réel)
Modèle	OpenAI (famille GPT)	Gemini (essais seulement)

Table 14 – Technologies retenues.

9.2 Description de l'interface : A2UI

Choix : l'agent décrit l'interface en A2UI (section 1.1), à partir d'un catalogue fermé de composants bancaires développés par l'équipe (carte de compte, liste de transactions, confirmation de virement…).

- **Garde-fou natif.** Le modèle ne produit aucun code exécuté par le navigateur ; un composant absent du catalogue ne peut pas être affiché. C'est la réponse la plus directe au pilier « génération dynamique vs. cadrage » du sujet, et au respect d'un Design System.
- **Composition à la demande.** Contrairement à l'approche statique, où chaque affichage doit avoir été prévu, l'agent assemble les composants selon la demande.
- **Intégration à la chaîne retenue.** A2UI est transporté par AG-UI et rendu nativement par CopilotKit ; l'assemblage a été démontré dans l'exemple d'architecture.
- **Mises à jour ciblées.** Les données sont séparées de la structure : l'agent peut mettre à jour un solde sans renvoyer l'interface.

Non retenu : OpenUI suit la même logique de catalogue et a fonctionné dans son essai, mais il ne s'insère pas dans la chaîne AG-UI et CopilotKit essayée, et sa consigne générée coûte environ un millier de tokens par appel (section 5). Il reste une alternative crédible. L'approche ouverte (HTML fourni par l'agent) offre trop peu de contrôle pour une banque.

Point d'attention : la spécification est en version 0.9, la 1.0 étant en cours de finalisation ; des changements sont possibles.

9.3 Agent – interface : AG-UI

Choix : la page et l'agent communiquent par AG-UI (section 1.2).

- **Fait pour les agents.** Les événements typés (texte en streaming, appels d'outils, état, interactions de l'utilisateur) couvrent ce qu'une simple requête HTTP ne permet pas : une exécution longue, au parcours imprévisible.
- **Indépendant du framework d'orchestration.** L'interface ne connaît que AG-UI : on peut changer de framework, ou de langage pour l'agent, sans la modifier. LangChain, Mastra ou Pydantic AI le prennent en charge.

— **Base de CopilotKit.** Le protocole est porté par l'éditeur de CopilotKit, qui l'utilise pour toute sa communication.

Non retenu : le flux propre à l'AI SDK de Vercel, qui fonctionne mais lie l'interface à cette bibliothèque.

9.4 Agent – outils : function calling et MCP

Choix : les outils bancaires (consulter les comptes, lister les transactions, préparer un virement) sont regroupés dans un serveur MCP (section 2.2), côté serveur ; l'agent les découvre et les appelle, le modèle les demandant par function calling (section 2.1).

— **Function calling : le mécanisme de base.** Il n'y a pas de choix à faire : c'est la façon dont les API de modèles, celle d'OpenAI comprise, permettent de demander un outil, et le framework d'orchestration le gère.
— **Outils côté serveur.** L'agent s'exécute sur le serveur, comme l'API bancaire : placer les outils dans la page imposerait un aller-retour par le navigateur. Côté serveur, l'application contrôle exactement ce que l'agent peut faire et journalise chaque appel.
— **MCP : des outils définis une seule fois.** Les outils sont partagés par tous les agents, y compris ceux qui pourraient s'ajouter (section suivante). L'essai a montré qu'un agent LangChain les découvre au démarrage sans en définir aucun.

Non retenu : WebMCP s'adresse aux agents intégrés au navigateur (section 2.3), alors que notre agent s'exécute côté serveur ; il pourra être ajouté plus tard, au-dessus des mêmes outils, si la banque veut rendre son site utilisable par ces agents. UTCP reste peu répandu.

Point d'attention : la spécification MCP évolue (version du 28 juillet 2026, sans état) et les SDK suivent avec un décalage : le SDK Python essayé négociait encore la version 2025-11-25. Pour un agent unique, le serveur MCP est un service de plus à faire tourner.

9.5 Agent – agent : A2A, si nécessaire

Choix : l'architecture démarre avec un seul agent. Si des agents spécialisés sont ajoutés (crédit, épargne…), l'agent assistant leur délèguera les demandes par A2A (section 3.1), en restant le seul interlocuteur de l'interface.

— **Standard ouvert, déjà essayé.** Chaque agent publie une carte décrivant ses compétences, et les tâches suivent des états définis ; l'essai a validé ce fonctionnement.
— **Complémentaire de MCP.** MCP relie un agent à ses outils, A2A relie des agents entre eux : les agents spécialisés utiliseraient les mêmes outils par le même serveur MCP.

Point d'attention : ajouter des agents augmente le nombre d'appels au modèle, donc la latence et le coût ; ce n'est justifié que si un domaine le demande.

9.6 Framework front-end : React

Choix : React (section 4), avec Vite comme outil de construction.

— **Le framework que l'équipe connaît le mieux**, ce qui réduit le temps de prise en main et laisse se concentrer sur l'interface générative elle-même.

— **Seul framework pris en charge par toutes les bibliothèques d'interface générative** étudiées, CopilotKit compris.
— **Écosystème le plus large** (environ 251k étoiles et 188 millions de téléchargements npm hebdomadaires), et framework déjà prévu par le dépôt et utilisé dans tous les essais.
— **Vite suffit** : le serveur (agent, API) est un composant séparé. Next.js n'apporterait un avantage que pour héberger l'agent ou le runtime CopilotKit dans la même application.

Non retenu : Vue, Angular et Svelte restent possibles mais réduisent le choix de bibliothèques.

9.7 Interface générative : CopilotKit

Choix : CopilotKit (section 5) affiche la conversation et rend les surfaces A2UI avec les composants du catalogue.

— **Toute la chaîne, essayée.** CopilotKit est fondé sur AG-UI et rend A2UI par son runtime ; c'est la seule bibliothèque avec laquelle l'assemblage AG-UI, A2UI et LangChain a été essayé. assistant-ui annonce aussi un adaptateur AG-UI et un rendu A2UI, mais son essai l'a relié à l'AI SDK.
— **Les trois approches dans un même outil** : composants associés aux outils (statique), surfaces A2UI (déclarative) et MCP Apps (ouverte), ce qui laisse la porte ouverte selon les cas d'usage.
— **Maturité.** C'est le plus suivi des frameworks dédiés (environ 38,1k étoiles), et il est indépendant du framework d'orchestration.

Non retenu : l'AI SDK UI a fonctionné, mais en approche statique et avec son propre flux plutôt qu'AG-UI ; assistant-ui, essayé de la même façon, est moins suivi (environ 12,6k étoiles) et sa page était plus lourde (274 ko compressés contre 116 pour l'AI SDK) ; OpenUI a son propre langage d'interface ; Tambo impose son propre agent, ce qui exclut LangChain.

Point d'attention : CopilotKit ajoute un *runtime* Node, relais entre la page et l'agent, soit un service de plus.

9.8 Orchestration : LangChain

Choix : LangChain (section 6), en Python.

— **Écosystème le plus mature** (environ 147,6k étoiles pour le dépôt Python), avec LangGraph pour les déroulements imposés et LangSmith pour l'observabilité.
— **Indépendant du modèle.** Le même code fonctionne avec GPT, Claude ou Gemini : passer de Gemini, utilisé pour les essais, au modèle d'OpenAI ne change que le nom du modèle.
— **Middlewares utiles au projet**, notamment la validation humaine avant un virement et le masquage des données personnelles.
— **Intégration démontrée** avec AG-UI, A2UI et CopilotKit dans l'exemple d'architecture, et avec MCP dans l'essai dédié.
— **Performances équivalentes** aux autres frameworks (tableau 9).

Langage : Python plutôt que TypeScript. La piste initiale était TypeScript, pour n'avoir qu'un langage avec l'interface. Les deux versions ont donné les mêmes résultats aux mesures (tableau 9), mais trois points ont conduit à retenir Python.

1. **Maturité de la version TypeScript.** Le chiffre de 147,6k étoiles concerne le dépôt Python. Le dépôt TypeScript (`langchain-ai/langchainjs`) en compte environ 18,3k, soit moins que Mastra (28,7k) ou que l'AI SDK de Vercel (27,4k). En téléchargements npm hebdomadaires (semaine du 5 au 11 octobre 2026), le paquet `langchain` en totalise environ 3,1 millions, `@mastra/core` 2,2 millions et `ai` (Vercel) 31 millions.
2. **Intégration avec CopilotKit.** Selon la documentation de LangChain, un agent LangChain.js se connecte à CopilotKit par l'intermédiaire d'un serveur LangGraph (serveur de développement ou déploiement LangSmith), et la page consultée ne mentionne pas A2UI. En Python, l'exemple d'architecture expose directement l'agent en AG-UI avec FastAPI. Ce point n'a pas été essayé en TypeScript.
3. **Cohérence avec le backend.** Le backend bancaire prévu est en Python (FastAPI, PostgreSQL). Un agent TypeScript ajoute un second langage côté serveur ; un agent Python le partage avec le backend.

S'y ajoutent la simplicité du langage, que l'équipe connaît bien, et la richesse de son écosystème de bibliothèques, notamment de calcul et de traitement de données. Le seul avantage de TypeScript, un langage unique avec l'interface, ne tient pas puisque le backend reste en Python.

Option	Pour	Contre
LangChain (TypeScript)	Un seul langage avec l'interface ; middlewares de validation humaine et de données personnelles ; mêmes concepts qu'en Python.	Écosystème TypeScript moins suivi que le Python ; connexion à CopilotKit par un serveur LangGraph, non essayée.
LangChain (Python)	Écosystème le plus répandu ; même langage que le backend ; intégration AG-UI, A2UI et CopilotKit déjà démontrée par l'exemple.	Deux langages dans le projet (TypeScript pour l'interface).

Table 15 – Langage de l'agent LangChain : Python retenu.

Non retenu : Mastra, CrewAI, Pydantic AI, les SDK des éditeurs de modèles et les autres frameworks mesurés offrent des performances équivalentes, mais un écosystème moins large ou une dépendance plus forte à un fournisseur de modèle.

9.9 Observabilité : LangSmith

Choix : LangSmith, dans son offre gratuite, pour le prototype (section 7).

— **Intégration native à LangChain** : il s'active sans modifier le code de l'agent.
— **Gratuit pour le prototype** : 5 000 traces par mois, largement suffisant pour le développement et la démonstration.
— **Données fictives** : envoyer les traces du prototype à un service hébergé ne pose pas de problème de confidentialité.

Pour un usage réel : les traces contiennent les questions des clients et les données bancaires ; elles devraient rester chez la banque, avec LangSmith en offre Enterprise ou une solution auto-hébergée comme Langfuse ou Phoenix. L'essai Phoenix a montré qu'un agent LangChain peut être tracé entièrement en local, sans modification de son code.

Organisation : l'offre gratuite ne comptant qu'un utilisateur, l'équipe utilisera un compte unique, ce qui suffit pour un prototype. L'offre Plus (39 dollars par utilisateur et par mois) ne

serait utile qu'avec des comptes individuels.

9.10 Modèle : OpenAI

Choix : un modèle d'OpenAI (famille GPT, celle de ChatGPT).

- **Accès fourni par le partenaire.** Le sujet prévoit que Sopra Steria fournisse l'accès aux modèles et les quotas ; le modèle exact et son mode d'accès sont à confirmer avec lui.
- **Changement sans impact sur l'architecture.** LangChain étant indépendant du fournisseur, passer de Gemini au modèle d'OpenAI ne modifie ni l'interface ni les outils.

Remarque : Gemini n'a servi qu'aux essais. Les mesures de la section 6 visaient à comparer les frameworks entre eux et donnent un ordre de grandeur ; le changement de modèle ne remet pas ces comparaisons en cause.

10 Conclusion

Cet état de l'art a passé en revue les technologies nécessaires à une application bancaire dont l'interface est générée par un agent : les protocoles qui relient l'agent à l'interface, à ses outils et aux autres agents, les frameworks front-end et d'interface générative, les frameworks d'orchestration et les outils d'observabilité. Chacune a été confrontée à sa documentation et, autant que possible, à un essai dans le dossier `experiments/` du dépôt.

Ce que montre l'étude. Les technologies étudiées ne sont pas concurrentes : chacune occupe une frontière précise du système (tableau 12). Les frameworks d'orchestration se valent sur les performances, avec le même modèle : tous réussissent les scénarios mesurés, en un temps dominé par l'appel au modèle. Les différences portent ailleurs : intégration avec les autres briques, maturité de l'écosystème, dépendance à un fournisseur et maîtrise des données. Plusieurs standards restent jeunes (A2UI en version 0.9, WebMCP expérimental, MCP dont la spécification évolue plus vite que ses SDK), ce qui invite à s'appuyer sur des bibliothèques suivies et à vérifier les versions.

L'architecture retenue. L'application repose sur un agent unique, côté serveur, écrit en Python avec LangChain. Il appelle les outils bancaires regroupés dans un serveur MCP, décrit les interfaces en A2UI à partir d'un catalogue fermé de composants, et diffuse son exécution par AG-UI vers une page React, où CopilotKit affiche la conversation et les composants. LangSmith assure l'observabilité du prototype, et le modèle sera fourni par OpenAI. Ces choix répondent aux quatre piliers du sujet :

- **Nouveaux patterns UX** : la conversation et les composants visuels coexistent dans la même interface, l'agent choisissant d'afficher une carte, une liste ou une demande de confirmation selon la demande.
- **Génération dynamique vs. cadrage** : l'agent compose l'interface, mais uniquement avec les composants du catalogue ; le modèle ne produit aucun code exécuté par le navigateur, et le Design System est respecté par construction.
- **Protocoles d'échange et outillage** : function calling, MCP, AG-UI et A2UI ont été essayés et s'assemblent ; A2A et WebMCP sont documentés et testés, prêts à être ajoutés si le besoin apparaît.

— **Industrialisation et robustesse** : l'agent n'agit qu'à travers des outils dont la liste est fixée par l'application, les actions sensibles passent par une confirmation de l'utilisateur, et chaque exécution peut être tracée.

Limites. Les mesures ont été réalisées avec un seul modèle, sur un agent simple à deux outils et avec des données fictives ; elles donnent un ordre de grandeur et comparent les frameworks entre eux, sans préjuger du comportement d'un agent plus riche. Certaines briques n'ont été que préparées ou documentées (Claude Agent SDK, Spring AI, Tambo, LangSmith, Langfuse), et l'exemple d'architecture a été réalisé avant le choix de placer les outils côté serveur : l'assemblage complet de l'architecture retenue reste à construire.

Perspectives. La suite du projet consiste à construire cette architecture sur le backend existant, puis à développer le catalogue de composants A2UI correspondant aux cas d'usage retenus avec le partenaire. Deux extensions sont préparées sans être nécessaires au démonstrateur : la délégation à des agents spécialisés par A2A, et l'ouverture du site aux assistants intégrés aux navigateurs par WebMCP, au-dessus des mêmes outils. Pour un usage réel, les traces de l'agent devraient enfin être conservées chez la banque, par une solution auto-hébergée comme celle essayée avec Phoenix.